

AI-Driven Customer Retention System

By: Shantanu Das

Contents

Acknowledgement	3
Introduction	4
The Project Setup	5
Problem Statement	5
Business Impact of Churn	5
Stakeholders	5
Tech Stack	5
Data Exploration and Analysis	7
Key Questions Before Analysis	7
Dataset Observation	7
Feature Relevance Validation	9
EDA Findings	11
Revenue Loss	19
Key Insights and Suggestions	20
Immediate Recommendations	22
Preparing Data for Modelling	23
Addressing The Main Concern	23
Importance of Using K-Fold Cross Validation	23
Data Preparation	24
Final Take	28
Model Building and Performance Evaluation	29
Helper Function	29
Models Under Consideration	31
Linear Models	32
Logistic Regression Without any Regularization	32
Logistic Regression with L2 Regularization at Different Regularization Strengths	33
Why Use Recall and F1 Score Instead of Accuracy	34
Logistic Regression With and Without Regularization Using SMOTENC	34
Logistic Regression With and Without Regularization Using SMOTEENN	37
Logistic Regression With and Without Regularization Using SMOTETomek	39
Comparison of Linear Models	41
Tree Based Models	42
Helper Function	42
Basic Decision Tree	44
Random Forest	49
XGBoost	52
LightGBM	53
CatBoost	55
Final Model	59
Training and Evaluation	59
The Business Impact	61
Feature Importance Revisited	62
Streamlit Prediction App with AI Generated Insights	63
The SHAP Features	63
The AI Generated Insights	63
The Streamlit App	66
Conclusion	74

Acknowledgement

This project marks more than just an academic milestone for me. It reflects a journey of learning, persistence, and growth, and I would like to take a moment to acknowledge the people who made it possible.

First and foremost, I am deeply grateful to my family and friends. Your constant support, patience, and belief in me have been my foundation through every high and low. Whether it was encouragement during tough phases or simply being there when I needed it, you've played a bigger role in this than words can capture. You kept me grounded and reminded me that I'm never really doing this alone.

I would also like to extend my sincere thanks to the faculty at Amity University and AlmaBetter. Your guidance, structured teaching, and willingness to clarify concepts have helped shape my understanding and approach toward data science and problem-solving. The clarity and discipline I've built through your mentorship are things I will carry forward.

I am equally thankful to my managers, team leads, and colleagues at ScribeEMR. Your support, professional guidance, and the environment you created played an important role in shaping my mindset and work ethic. The lessons I've taken from that experience go far beyond technical skills and have contributed significantly to how I approach challenges today.

A special acknowledgment goes out to the countless educators on YouTube and online platforms who share knowledge freely with the world. You may never know the faces you've impacted, but your content has been an invaluable part of my learning journey. In many ways, you've made quality education accessible without barriers, and that matters more than most people realize.

Finally, I want to acknowledge the process itself. The confusion, the trial and error, the small wins, and the occasional frustration. Every bit of it contributed to this outcome.

Even if this page goes unread, the gratitude behind it is very real.

Thank you.

Introduction

This is an AI-driven customer churn prediction system that identifies customers who are at risk of churning from the business, provides insights into the key drivers behind their behaviour, and recommends actions to improve retention.

Customer acquisition is expensive, making retention critical for telecom companies. By analysing customer behaviour and learning from previously churned users, businesses can identify at-risk customers early, take targeted action, and ultimately improve customer lifetime value. A 5% reduction in churn can save \$30,000 per month in revenue loss, in a company with 10,000 customers and an average monthly charge of \$60 per customer.

This project demonstrates the ability to tackle a real-world business problem and build an end-to-end system, from data preparation and feature engineering to analysis of the customer behaviour, identification of risk-factors, building and deploying predictive models, and translating outputs into actionable insights. It also highlights the ability to bridge technical analysis with business decision-making through explainability, AI-driven recommendations, and a business-friendly reporting layer.

The Project Setup

Problem Statement

A telecom company wants to identify customers at risk of churn and understand key trends and factors that drive this behaviour, enabling timely interventions through targeted actions to improve customer retention.

Business Impact of Churn

Customer churn has a very negative impact on business performance because:

- It reduces customer lifetime value with respect to acquisition cost, making it difficult to generate sustainable revenue.
- It increases cost as acquiring new customers is significantly more expensive than retaining existing ones.
- It weakens customer loyalty, reducing opportunities for returning business and organic growth through referrals.
- It signifies declining customer satisfaction and potential problems with service quality, pricing, or user experience.
- In a competitive market it directly affects market share as churned customers often move on to competitors.

Stakeholders

This system will provide actionable insights to:

1. Product Managers: Identify product-related issues and features that contribute to customer churn, enabling data-driven improvements.
2. Marketing Teams: Target high-risk customers with personalized retention strategies, reducing reliance on costly blanket campaigns.
3. Customer Success Teams: Understand key drivers of dissatisfaction and prioritize outreach to improve customer experience.
4. Leadership: Gain a high-level view of churn trends to support strategic decision-making and revenue planning.
5. Business Analysts: Analyse churn patterns and segmentation to support ongoing monitoring and reporting.

Tech Stack

- ❖ Excel
 - PowerQuery – For initial data preparation, cleaning and feature engineering.
 - Pivot Tables – For initial look into the data and identifying high-level trends.
 - Pivot Charts – Business-friendly visualizations for quick trend analysis and reporting.
 - SUMIFS, COUNTIFS, and Lookup Functions – To perform quick aggregations and look up desired values.
- ❖ Python
 - Pandas – For data cleaning, manipulation, summarization, EDA and preprocessing
 - NumPy – For numerical operations and feature handling
 - Matplotlib / Seaborn – For visualization and deeper EDA

- Scikit-learn – For data preprocessing, model building, and evaluation
- Imblearn – for SMOTE techniques used and building pipelines
- Xgboost – For implementation of XGBClassifier algorithm
- LightGBM – For implementation of LGBMClassifier algorithm
- Catboost – For implementation of CatBoostClassifier algorithm
- Pathlib and Sys – For directory related tasks
- Joblib – For saving and loading trained models
- Streamlit – For deploying the AI Driven Customer Retention System
- Groq – For the LLM layer using the "llama-3.3-70b-versatile" model
- ❖ Git and GitHub – For version control.

Data Exploration and Analysis

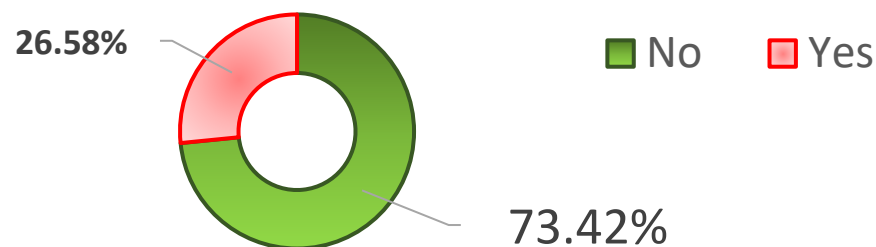
Key Questions Before Analysis

- How is churn defined and represented in the dataset?
- Is the target variable balanced or imbalanced?
- Which customer attributes are likely to be strong predictors of churn?
- Are there missing values, inconsistencies, or redundancies?
- Which features require statistical validation before model building?
- Are there weak features that may be excluded while modelling?

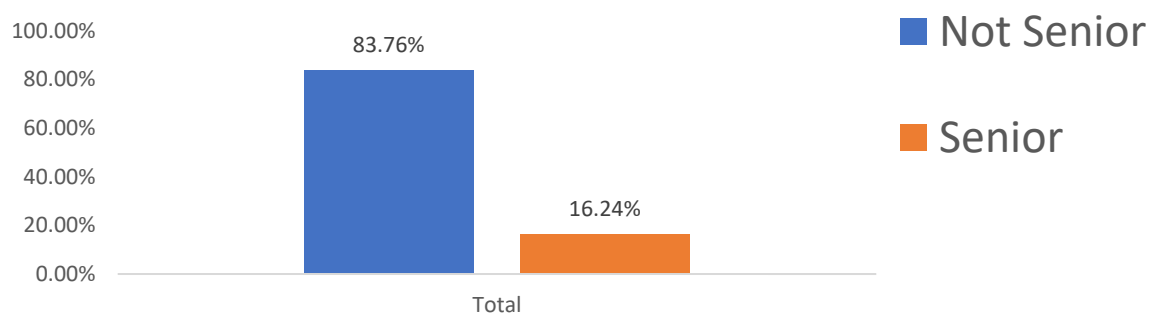
Dataset Observation

- ✓ Churn is defined by the company as the attrition of a customer, represented in the dataset as a binary target.

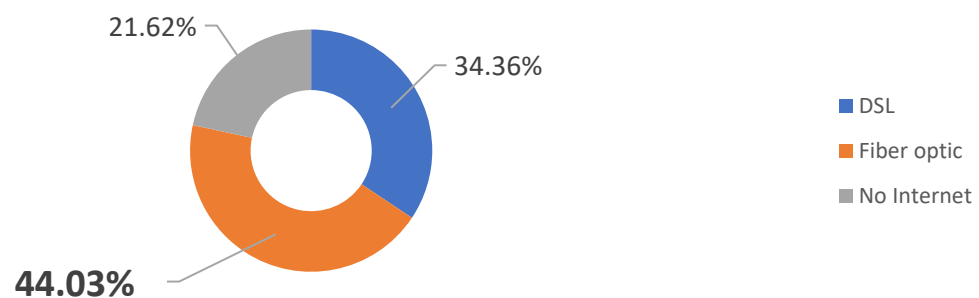
Proportion of Churned and Retained Customers



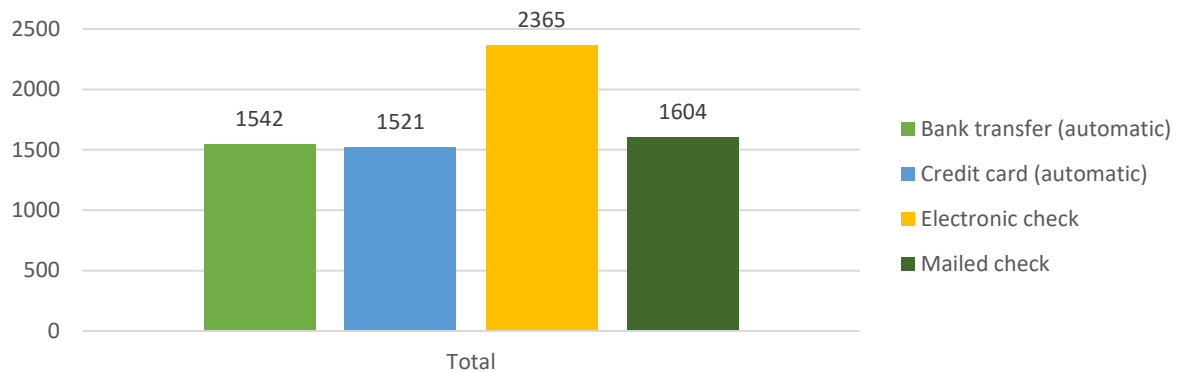
Proportion of Senior and Not Senior Customers



Proportions of Different Internet Services

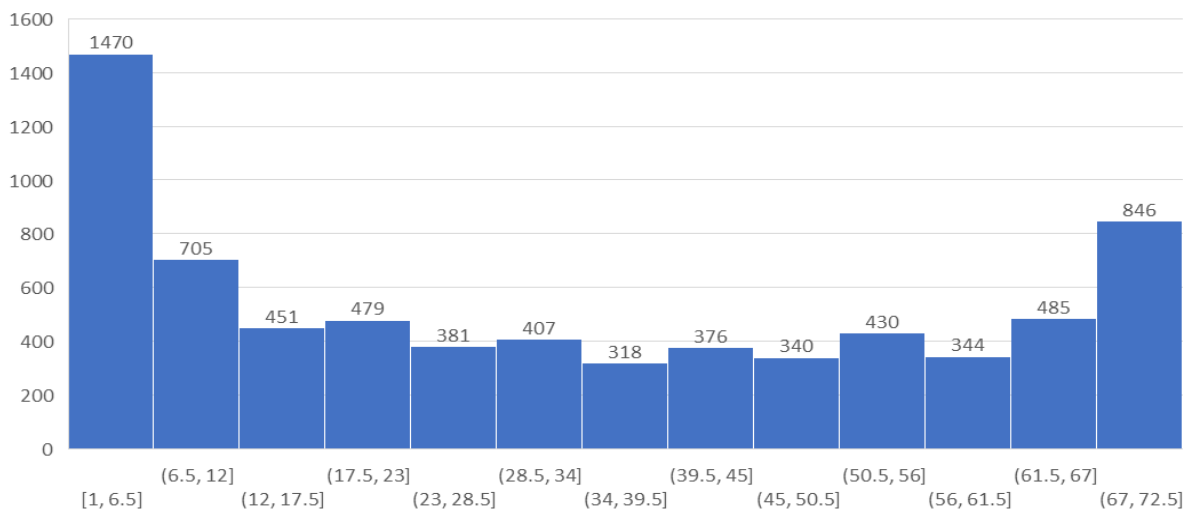


Count of Customers Using Different Payment Methods

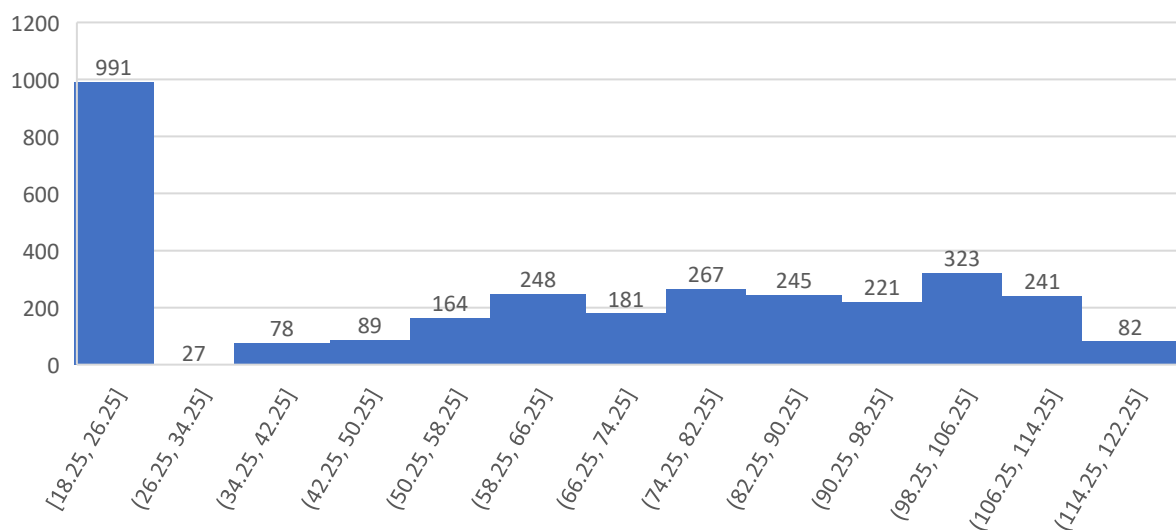


- ✓ The numerical variables such as tenure and monthly charges do not have a normal distribution.

Distribution of customers by tenure



Distribution of Monthly Charges



- ✓ There was a total of 11 missing values in the 'TotalCharges' column, which were dealt via power query and although there is no redundant data, there are inconsistencies such as 'SeniorCitizen' a binary column is labelled with 1 and 0 whereas other binary columns are labelled as 'Yes' or 'No'.

Feature Relevance Validation

- ✓ Chi-square test and Mann-Whitney-U test was used for hypothesis testing to find the presence of association of all categorical and numerical independent variables, respectively, with Churn. Later, Cramer's-V and Eta statistical test was performed to find the strength of the association. The table below depicts the findings.

Null Hypothesis	Alternate Hypothesis	Test	p-value	Result	Verdict	Strength of Association
Contract is not related to churn	Contract is related to churn	Chi-Square	7.3262E-257	Null Hypothesis is Rejected	Contract is related to churn	0.40956
Tenure is not related to churn	Tenure is related to churn	Chi-Square	4.12E-192	Null Hypothesis is Rejected	Tenure is related to churn	0.354108
Online Security is not related to churn	Online Security is related to churn	Chi-Square	1.4007E-184	Null Hypothesis is Rejected	Online Security is related to churn	0.346992
Tech Support is not related to churn	Tech Support is related to churn	Chi-Square	7.4078E-180	Null Hypothesis is Rejected	Tech Support is related to churn	0.342506
Internet Service is not related to churn	Internet Service is related to churn	Chi-Square	5.8312E-159	Null Hypothesis is Rejected	Internet Service is related to churn	0.321909
Payment Method is not related to churn	Payment Method is related to churn	Chi-Square	1.4263E-139	Null Hypothesis is Rejected	Payment Method is related to churn	0.30296
Online Backup is not related to churn	Online Backup is related to churn	Chi-Square	7.7761E-131	Null Hypothesis is Rejected	Online Backup is related to churn	0.291902
Device Protection is not related to churn	Device Protection is related to churn	Chi-Square	1.9594E-121	Null Hypothesis is Rejected	Device Protection is related to churn	0.281159
Streaming Movies is not related to churn	Streaming Movies is related to churn	Chi-Square	5.35356E-82	Null Hypothesis is Rejected	Streaming Movies is related to churn	0.230702
Streaming TV is not related to churn	Streaming TV is related to churn	Chi-Square	1.32464E-81	Null Hypothesis is Rejected	Streaming TV is related to churn	0.230143
Total Charges is not related to churn	Total Charges is related to churn	Mann-Whitney-U	2.00E-84	Null Hypothesis is Rejected	Total Charges is related to churn	0.199484
Monthly Charges is not related to churn	Monthly Charges is related to churn	Mann-Whitney-U	8.47E-54	Null Hypothesis is Rejected	Monthly Charges is related to churn	0.192858
Paperless Billing is not related to churn	Paperless Billing is related to churn	Chi-Square	5.29185E-58	Null Hypothesis is Rejected	Paperless Billing is related to churn	0.191454
Dependents is not related to churn	Dependents is related to churn	Chi-Square	1.3468E-42	Null Hypothesis is Rejected	Dependents is related to churn	0.163128
Senior Citizen is not related to churn	Senior Citizen is related to churn	Chi-Square	1.55857E-36	Null Hypothesis is Rejected	Senior Citizen is related to churn	0.150541
Partner is not related to churn	Partner is related to churn	Chi-Square	2.82424E-36	Null Hypothesis is Rejected	Partner is related to churn	0.149982
Billing Gap is not related to churn	Billing Gap is related to churn	Chi-Square	1.67E-06	Null Hypothesis is Rejected	Billing Gap is related to churn	0.061512
Multiple Lines is not related to churn	Multiple Lines is related to churn	Chi-Square	0.003567927	Null Hypothesis is Rejected	Multiple Lines is related to churn	0.040036
Gender is not related to churn	Gender is related to churn	Chi-Square	0.473665182	Failed to Reject Null Hypothesis	Gender is not related to churn	0
Phone Service is not related to churn	Phone Service is related to churn	Chi-Square	0.326885807	Failed to Reject Null Hypothesis	Phone Service is not related to churn	0

- ✓ As per the table above, we can see that:
 - The columns – Contract, Tenure, OnlineSecurity, TechSupport, InternetService, and PaymentMethod have strong association with Churn.
 - The columns – BillingGap, MultipleLines, Gender, and PhoneService have negligible association with Churn.
 - The remainder of the columns have a moderate association with churn.

- ✓ Using the Variance Inflation Factor by choosing one feature at a time and regressing all others against it, the following table was found which shows that MonthlyCharges, InternetService, PhoneService, StreamingMovies, StreamingTV, and TotalCharges have very strong multicollinearity. Tenure, MultipleLines, DeviceProtection, OnlineBackup, TechSupport, and OnlineSecurity, have strong multicollinearity. Whereas the other remaining features have low to moderate multicollinearity.

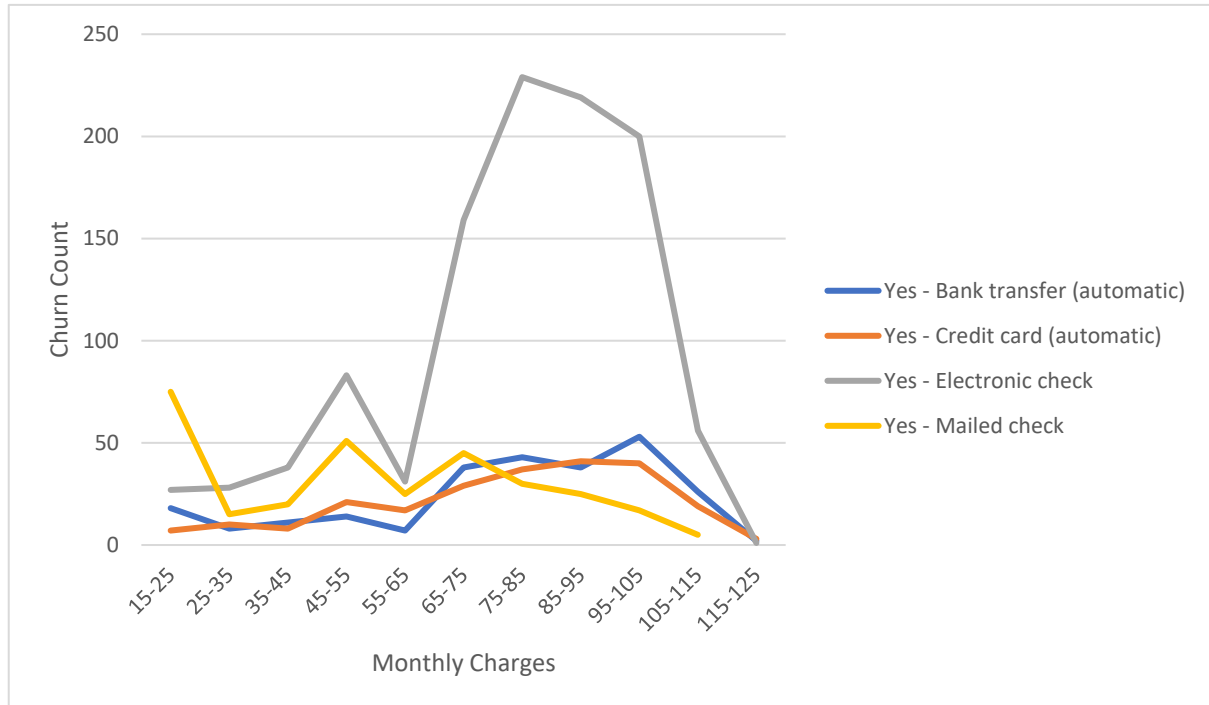
Feature	vif_score	Multicollinearity
MonthlyCharges	866.0227905	Vert Strong
InternetService_Fiber optic	591.9664851	Vert Strong
InternetService_DSL	138.6240921	Vert Strong
PhoneService	34.89199863	Vert Strong
StreamingMovies	24.14714862	Vert Strong
StreamingTV	24.06465389	Vert Strong
TotalCharges	10.81125075	Vert Strong
Tenure	7.579690055	Strong
MultipleLines	7.286034217	Strong
DeviceProtection	6.924695546	Strong
OnlineBackup	6.794867704	Strong
TechSupport	6.476330786	Strong
OnlineSecurity	6.33621707	Strong
Contract_Month-to-month	3.579740813	Low - Moderate
PaymentMethod_Electronic check	2.174156346	Low - Moderate
PaymentMethod_Bank transfer (automatic)	1.803411677	Low - Moderate
PaymentMethod_Credit card (automatic)	1.774047793	Low - Moderate
Contract_One year	1.712020108	Low - Moderate
Partner	1.462982006	Low - Moderate
Dependents	1.38104528	Low - Moderate
SeniorCitizen	1.15152956	Low - Moderate
Gender_Male	1.0020207	Low - Moderate

- ✓ After removing the MonthlyCharges and TotalCharges, the VIF values come down drastically as shown in the table below. The values are between 1-5 which show low-moderate multicollinearity, which is still considered safe to model. However, feature validation will be done once more during the model building process.

Feature	vif_score	Multicollinearity
InternetService_Fiber optic	3.976395624	Low-Moderate
Contract_Month-to-month	3.55404804	Low-Moderate
InternetService_DSL	3.54299179	Low-Moderate
Tenure	2.824158888	Low-Moderate
PaymentMethod_Electronic check	2.132441533	Low-Moderate
PaymentMethod_Bank transfer (automatic)	1.784716738	Low-Moderate
PaymentMethod_Credit card (automatic)	1.760407281	Low-Moderate
Contract_One year	1.697232973	Low-Moderate
StreamingMovies	1.631957242	Low-Moderate
StreamingTV	1.619443692	Low-Moderate
TechSupport	1.481296685	Low-Moderate
DeviceProtection	1.479255385	Low-Moderate
Partner	1.46236329	Low-Moderate
MultipleLines	1.422169602	Low-Moderate
OnlineSecurity	1.410764593	Low-Moderate
OnlineBackup	1.380395213	Low-Moderate
Dependents	1.380261048	Low-Moderate
PhoneService	1.354685399	Low-Moderate
SeniorCitizen	1.151479274	Low-Moderate
Gender_Male	1.001680234	Low-Moderate

EDA Findings

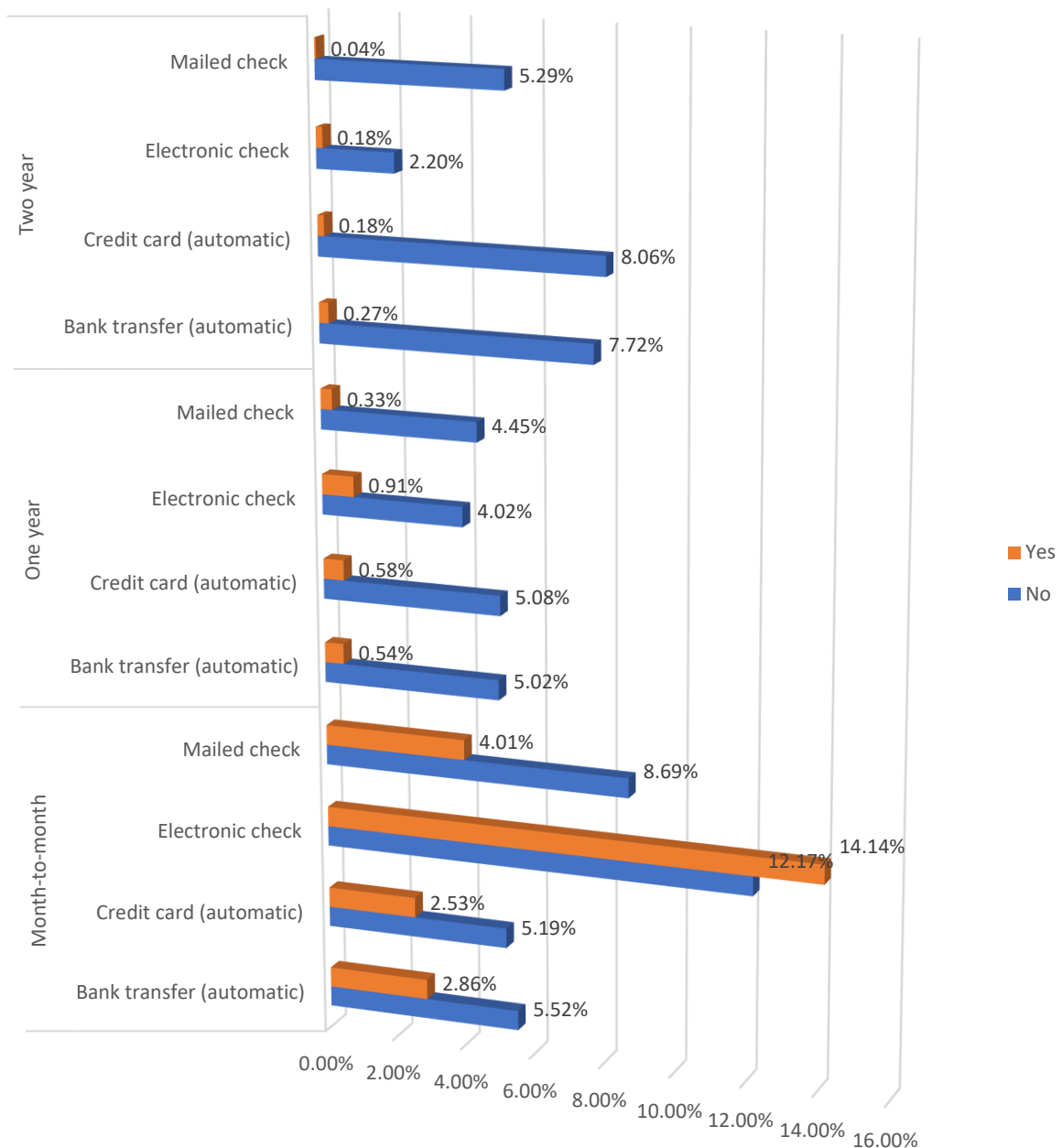
- ❖ Customers with plans that charge between 65 and 110 and paying via electronic check, on a month-to-month basis, have some of the highest attrition whereas customers with monthly charges of 10-40 USD paying via credit card have some of the highest retention.



- In fact, high monthly charges and paying via electronic check individually have high churn as well, indicating that:
 - Customers are not satisfied with what they receive for the price they are paying.
 - There might be some fault in the electronic check payment system, such as failure rates or high charges for the payment itself.

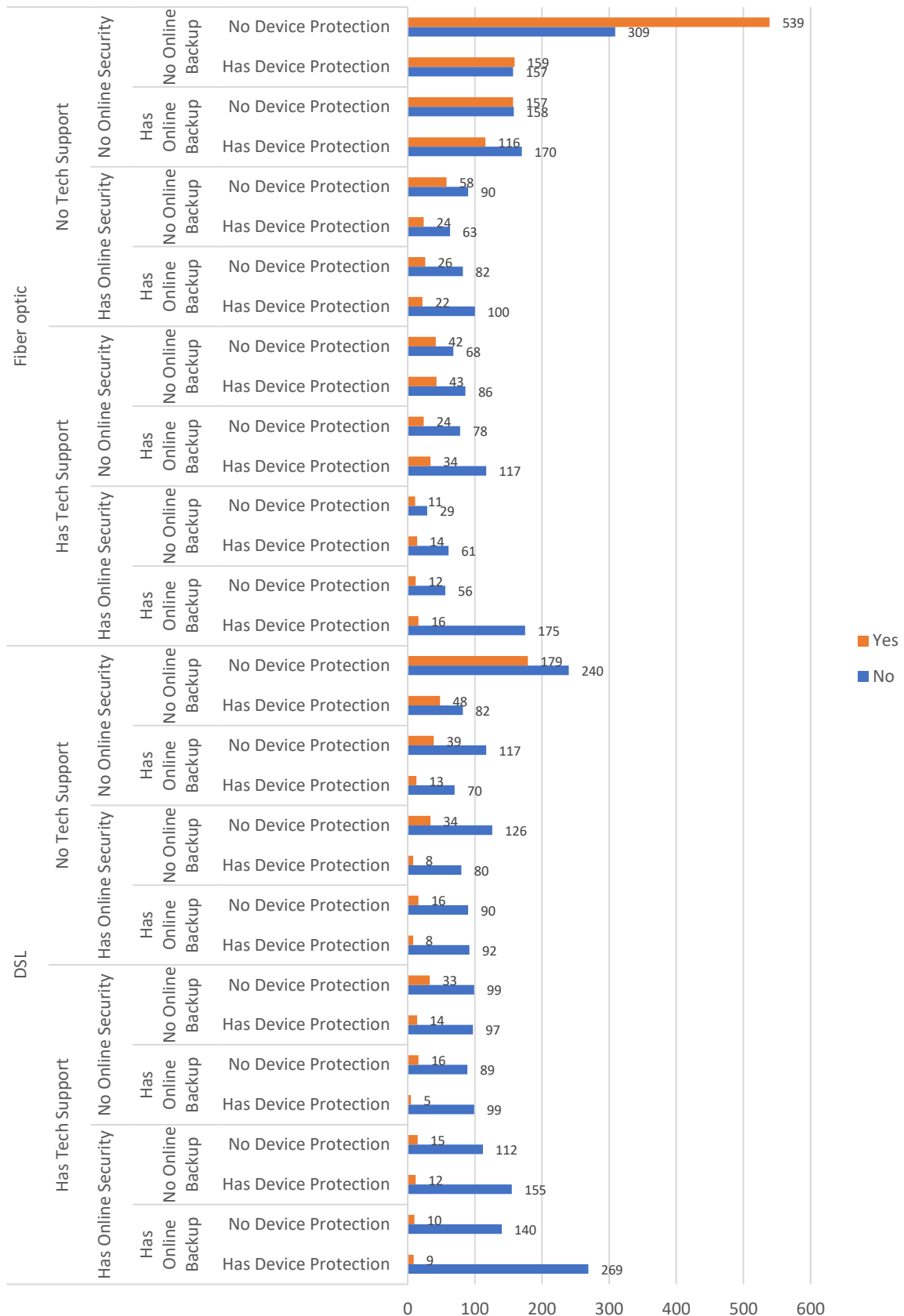
This is further reinforced by the ChiSquare-CramersV and MannWhitneyU-Eta finding during the hypothesis testing which showed that MonthlyCharges has a moderately strong association with Churn and PaymentMethod has a Strong association with Churn.

- ✓ Immediate suggested actions:
 - Personalized mini survey sent via mail/SMS (or Whatsapp if registered), to customers with high monthly charges to see if they are satisfied with the service and products.
 - Personalized feedback message sent via mail/SMS (or Whatsapp if registered), or a feedback page that pops up after making payment, that asks customers how their payment experience was and if it was seamless. An added requirement of a text section that further explains what the difficulty was will provide deeper insights into the problems being faced, so that they can be fixed quickly.
 - Customers with high monthly charges and paying via electronic check are already at risk and apart from above mentioned measures, providing them with occasional discounts on charges and discounts on bundled services might bring down the chances of churn.

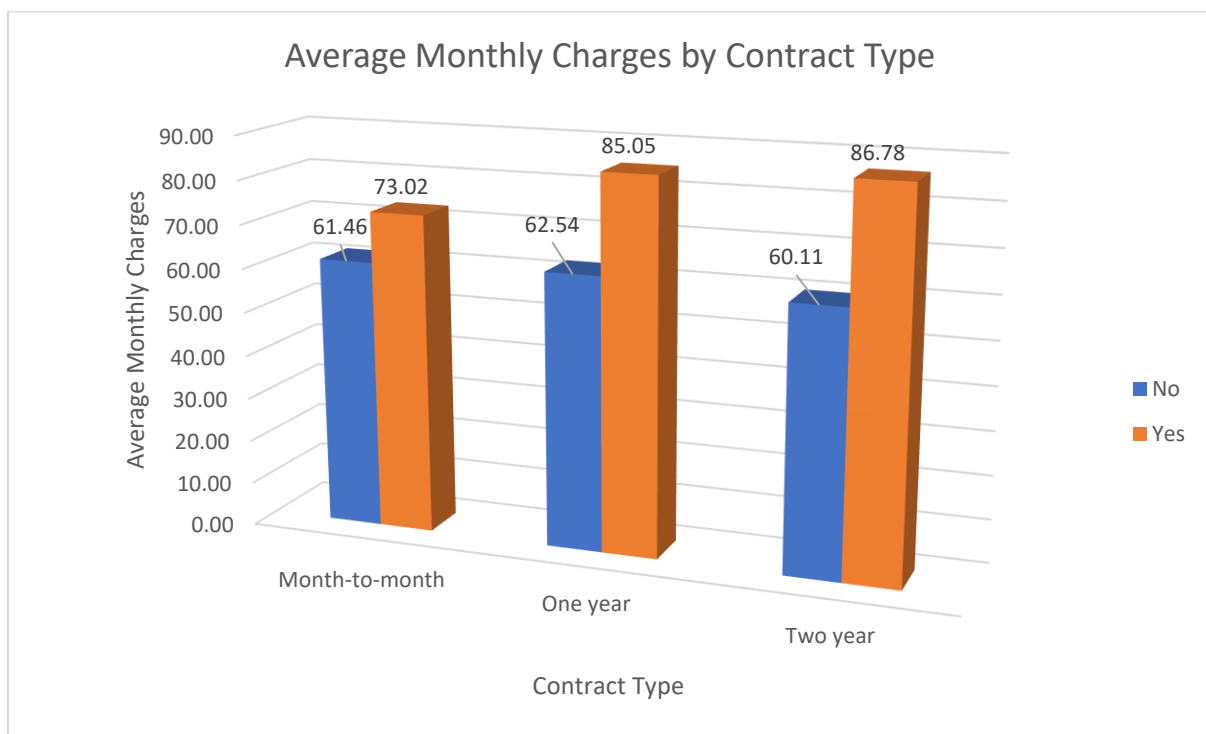


- ❖ The chart above shows that the month-to-month contract type generally has the highest rate of churn, especially among customers who are paying via electronic check. This fact is further strengthened by the fact that during hypothesis testing via ChiSquare and CramersV, it was found that Contract has the highest association with Churn among all other features.
- ✓ Customers on a month-to-month contract type should be warm called or sent upsell emails, with discounted prices on one- and two-year plans, especially if bundled with other services, and explained on the benefits, to help address dissatisfaction and eventual Churn.

- ❖ Fiber optic customers without added services such as Online Security, Online Backup, Device Protection and Technical Support have some of the highest churn. This is further supported by the Hypothesis Testing as InternetService has a strong association with Churn.

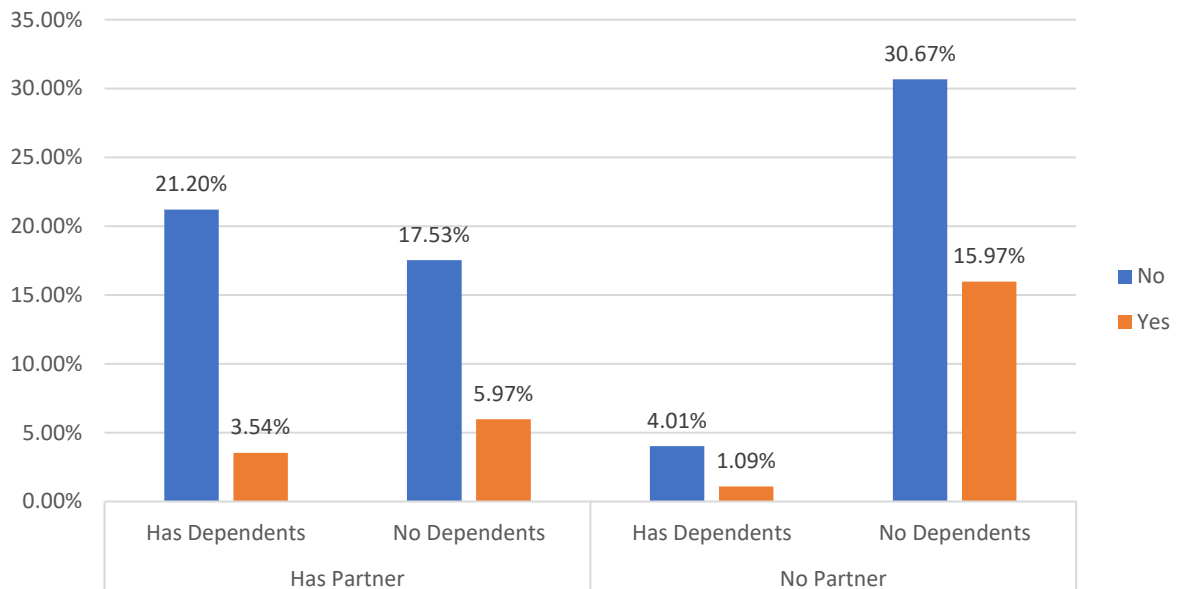


- ✓ Customers should be made aware of the benefits of different add-on services such as online security and device protection and individuals purchasing bundles of add-on services can be given a special discount, especially if paired with a one- to two-year contract plan.
- ❖ The average monthly charges of retained customers are comparatively lower than churned customers, and very similar across all contract types. But, the churned customers pay significantly different amounts across the contract types.
 - This shows some amount of mismatch between the customers expectation of the service that is to be provided, and the actual service being received. This most likely is due to add-ons being taken that customers do not actually need, and/or costlier plans, especially for fiber optic customers, who might not need such bandwidth or limits.



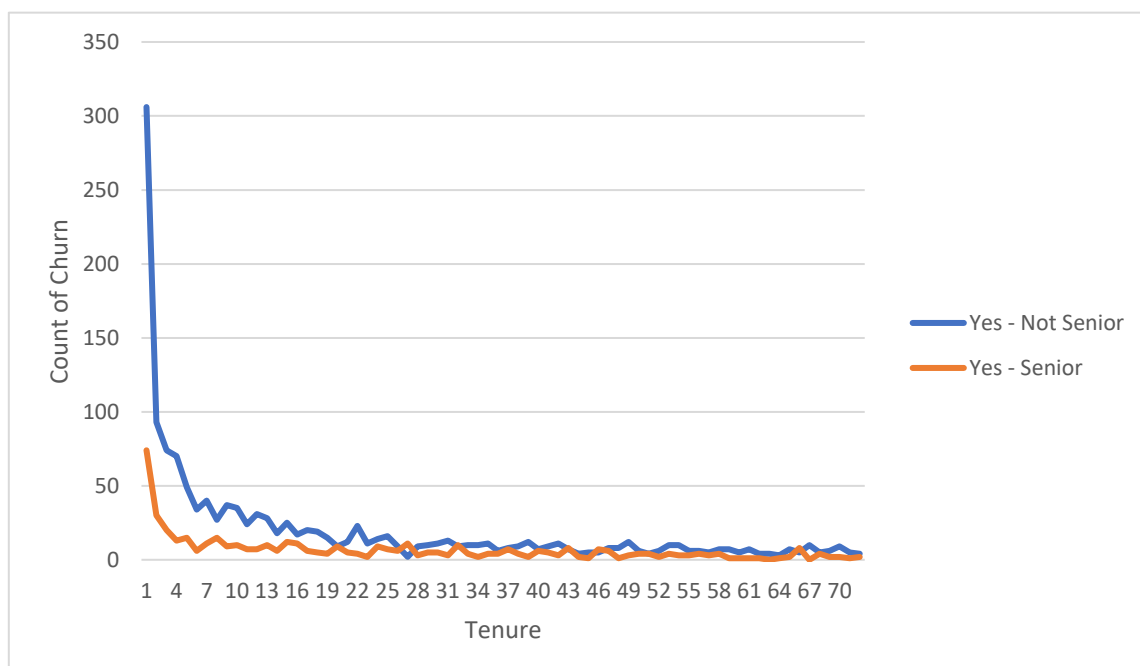
- ✓ Special technical training should be provided to the employees of the company who onboard the customers, so that they can know the products provided by the company better and by understanding the products better, they can better understand the needs of the customer, which in turn will help them guide the customers towards plans and add-ons relevant to them, so that overspending can be avoided on the customer's side and satisfaction can be increased.

- ❖ Customers without any partner or dependents have a high churn rate.
 - Hypothesis testing revealed that both Partners and Dependents, have mid-moderate association with churn, and although that does not explain the bulk of the churn, across both categories of partners, it is clear that not having dependents have a higher rate of churn. This most likely is due to single people realizing that they either do not need the service at all, or are overpaying for the service that they are receiving.

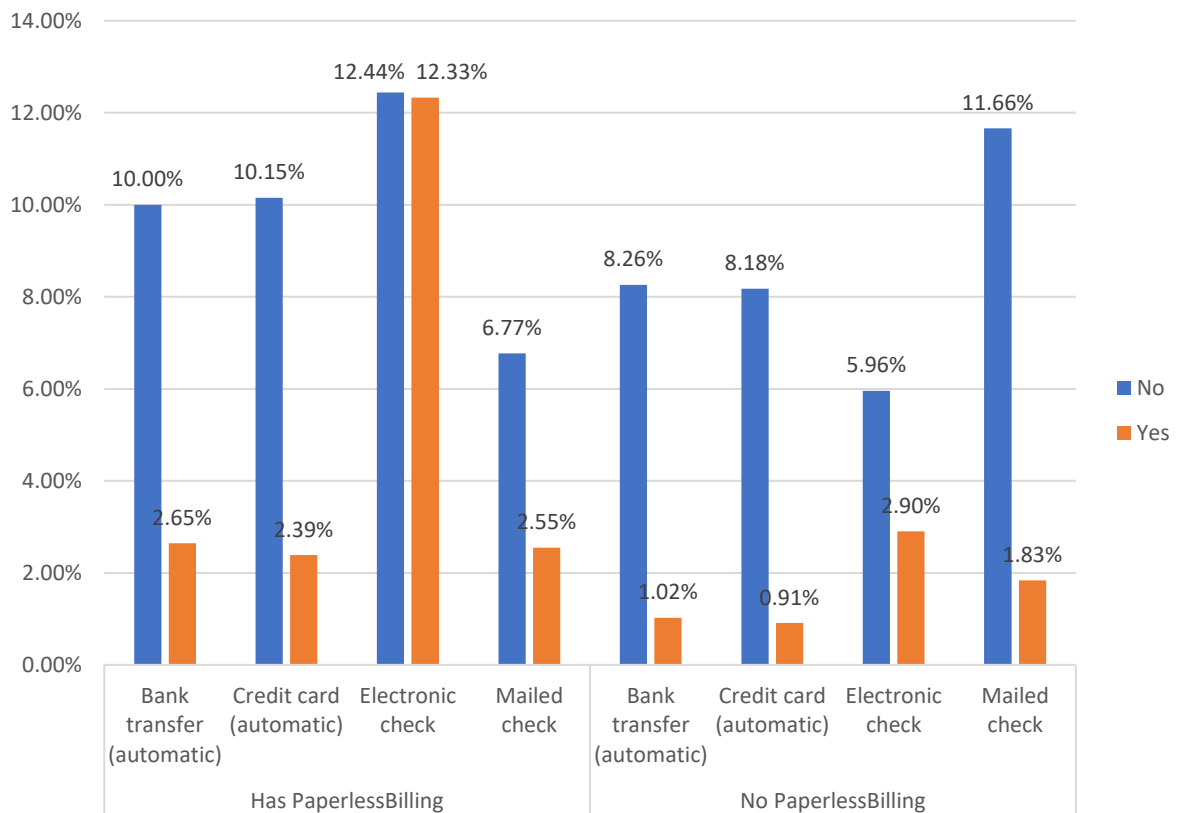


- ✓ This issue should be addressed by identifying the individuals without any partner or dependents from demographic information that is obtained during onboarding, and sending them a mini personalized survey as well, either via email or SMS/Whatsapp(if registered), asking whether they are satisfied so far and any concerning responses can be dealt with accordingly by reaching out and clarifying the issues being faced, so as to identify potential customers who are at risk early, and take corrective measures.

- ❖ Attrition rates are highest among younger people with less than 1 year of tenure and retention improves drastically for more tenured customers, stabilizing after 12-13 months.

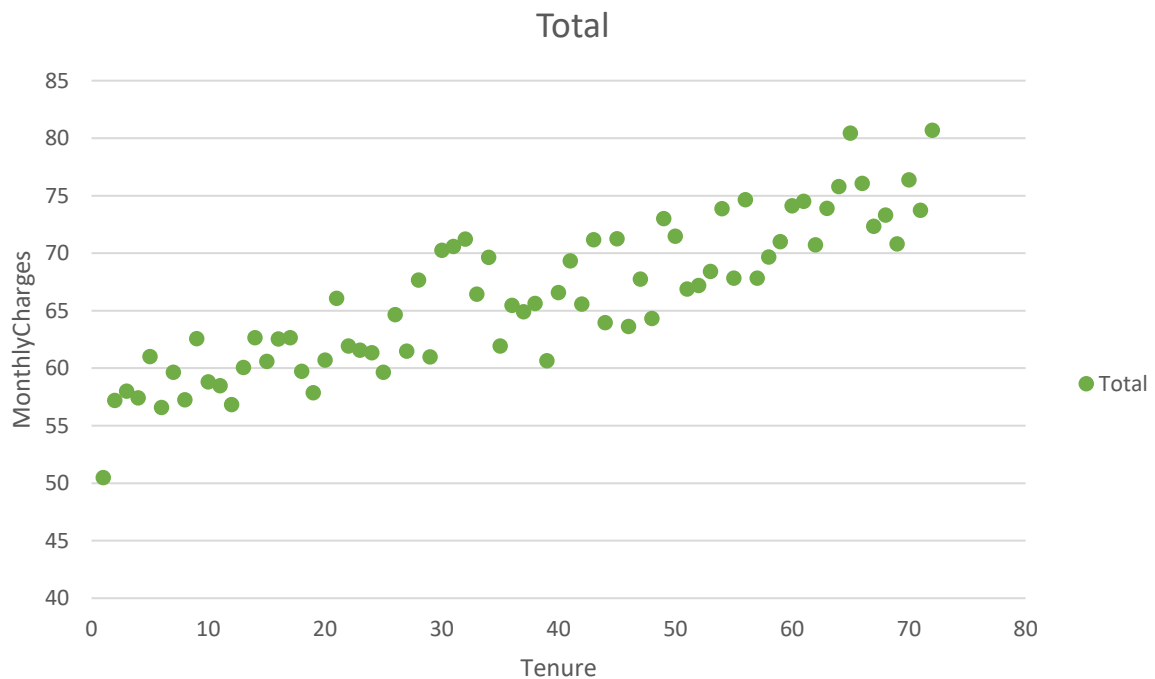


- ✓ Giving younger individuals a bonus with StreamingTV and StreamingMovies bundled with the service, if plans on the upper end are purchased and/or if one- or two-year contracts are made, then adding value added services, will most likely improve retention for the 1st year and within that time, the customer will be stabilized and used to the service, which will increase the chances of retention.
- ❖ Individuals paying via electronic check and who have availed paperless billing, have the worst churn rate.
 - Individuals who are environmentally friendly and/or prefer to be organized, will most likely opt for both paperless billing and electronic check payment. Since electronic check already has high churn, paperless billing will add to it if the bills are inconsistent, the billing is not done on time and/or bills are not sent to the respective places in time. This will increase the frustration among customers that will lead to customers Churning.

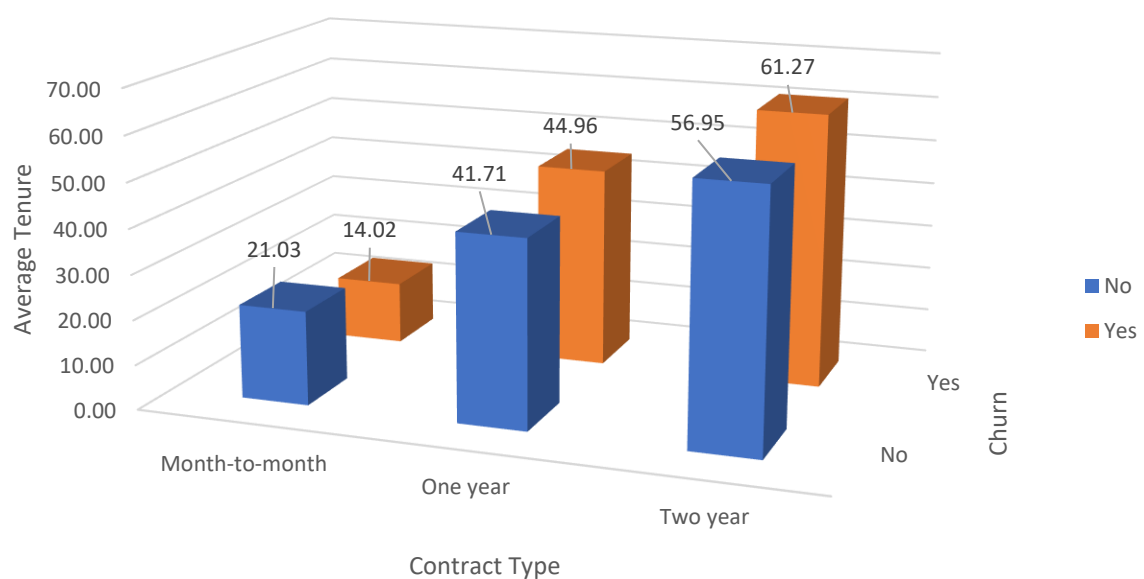


- ✓ Going over the entire billing system, automating as much of it as possible via machines that feed data into databases and agentic AI workflows that can create the bill structure and fill it accordingly, will lead to a reduction in human induced errors or inconsistencies in billing that will increase the satisfaction and reduce the chances of churn.

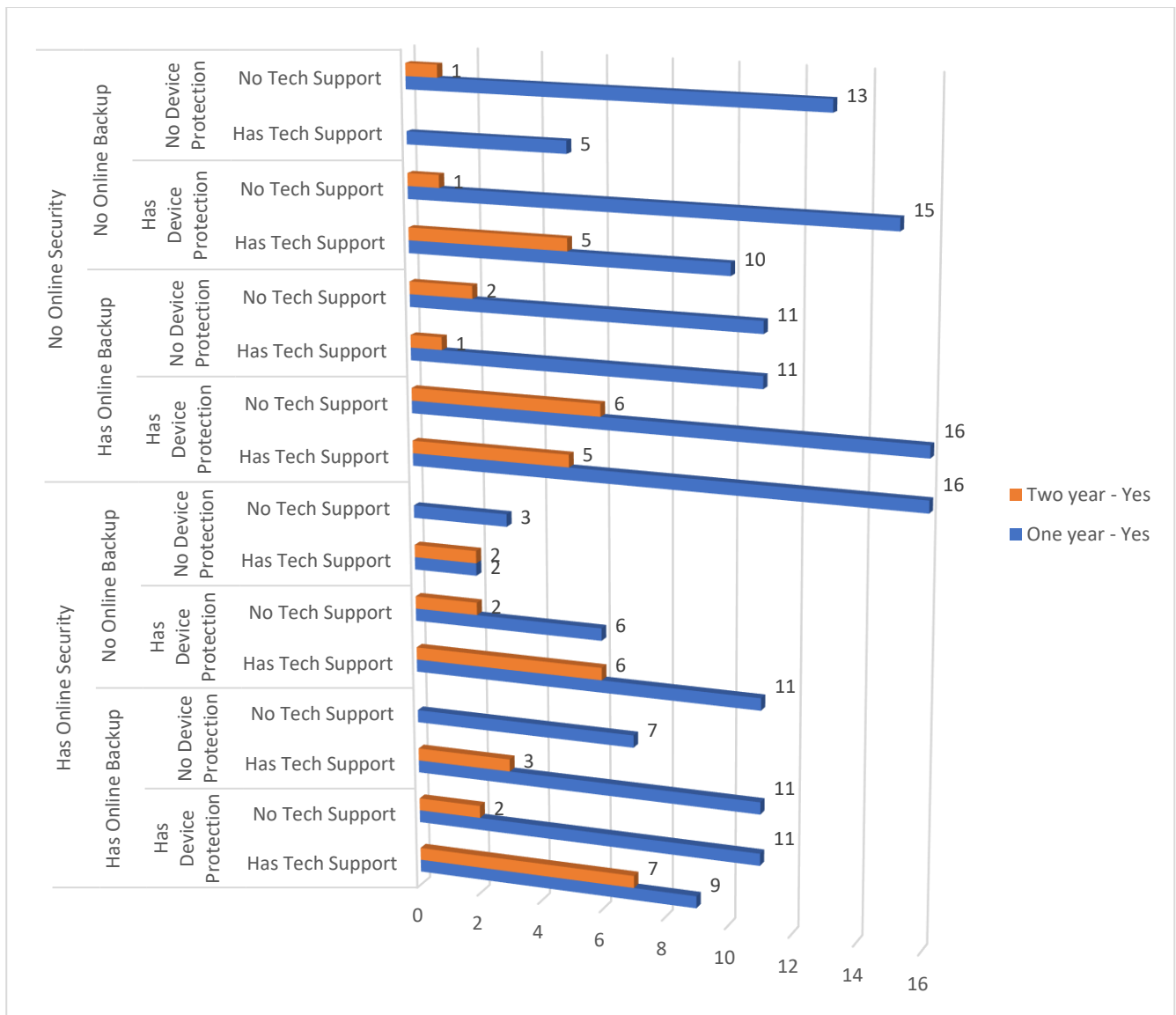
- ❖ The average monthly charges for customers increase with tenure, signalling an incentive to retain them as it will lead to organic growth of customer lifetime value.



- ❖ The average tenure of customers who have churned and had a month-to-month contract type was lower than retained customer, however, customers with one- and two-year plans who have churned have higher average tenure indicating that something went wrong which made them leave.



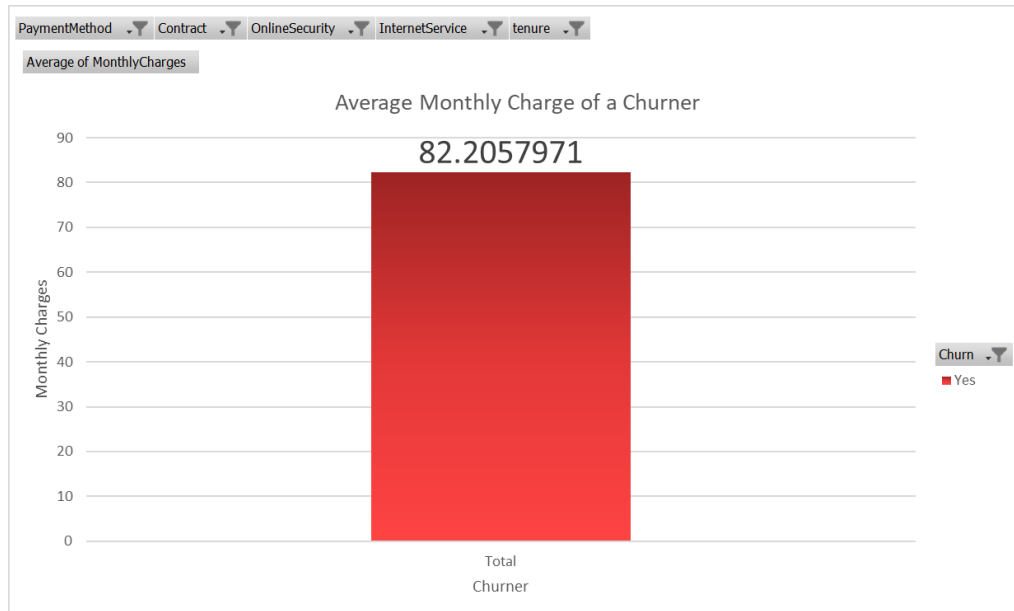
- ✓ This shows the importance of regular feedback calls or mails or messages so that the company can keep understanding the customer's point of view to identify areas where the customers is suffering and also identify the areas where the customer is satisfied. This will help the company to reduce the customer dissatisfaction and in turn increase the chances of retention.
- ❖ The chart below shows that among the customers with one- and two-year contracts who have churned, the majority are customers, although few in number, who did not avail the online security service.



- ✓ As mentioned in the last point, a timely survey or feedback might have revealed that customers are having problems with online security or are at least concerned about it which could have been used to bundle required services and offered at a discounted price to the customer which helps the plan feel personalized and will reduce Churn.

Revenue Loss

Using the above gained insights that the average profile of a churner is one who has high monthly charges, uses fiber optic internet, pays via electronic check, does not have online security, is low tenured, and is on a month-to-month contract, the average monthly charge of a churner was found to be ~\$82.



The total Revenue loss for the company was calculated to be \$139,130.



Key Insights and Suggestions

1. The dataset reveals an overall **Churn rate of 26.58%** and there exists **class imbalance** in the target variable (Churn), along with other categorical variables. The **numerical variables do not have a normal distribution** of its values.

Suggestion: Consider class imbalance while modelling the data, **use weighted classes or SMOTE variants** to address it. For the **numerical variables**, the dataset size (7032 after removal of missing values) is big enough that **CLT takes effect and the sampling distribution becomes normal**.

2. **Statistical testing** via
 - a. **ChiSquare** (for hypothesis testing regarding association with churn) and **Cramer's V** (for finding the strength of association where it exists) for categorical variables, and
 - b. **Mann-Whitney-U** (for hypothesis testing regarding association with churn) and **Eta** (for finding the strength of association where it exists) for numerical variables

Reveals that **Contract, Tenure, OnlineSecurity, TechSupport, InternetService, and PaymentMethod** have **strong association with Churn**, whereas, **MultipleLines, Gender, and PhoneService** have **negligible association with Churn**.

Also, via **Variance Inflation Factor (VIF)** testing, it was found that **MonthlyCharges** and **TotalCharges** had **high VIF values**, along with others, signalling that it is quite possible to predict them using the other features, **revealing multicollinearity**. **Removing** those two **features** brought the **VIF score** of all other features **down to the 1-5 range** which shows low to moderate multicollinearity, which is fine for modelling.

3. Since **Contract** and **OnlineSecurity** has **high association with churn**, and other add-on services like **DeviceProtection, OnlineBackup, and TechSupport** have **mild to moderate association** with churn, the following will help with retention.

Suggestion:

- a. Personalized mini survey sent via mail/SMS (or Whatsapp if registered), to customers with high monthly charges to see if they are satisfied with the service and products.
 - b. Customers can be made aware of the benefits of different add-on services such as online security and device protection and individuals purchasing bundles of add-on services can be given a special discount, especially if paired with a one- to two-year contract plan.
4. Along with the aforementioned, **Partner, Dependents** and **SeniorCitizen** also have mild-moderate association with Churn as not having a partner or dependents increases the chances of Churn, especially if we take **Tenure** into the picture, which has a **strong association with Churn**. This is because as seen earlier, younger people tend to churn more, especially before the completion of 1 year, which stabilizes after the 13th month onward.

Suggestion:

- a. Giving younger individuals a bonus with StreamingTV and StreamingMovies bundled with the service, if plans on the upper end are purchased and/or if one- or two-year contracts are made, then adding value added services, will improve retention for the 1st year.
- b. Identifying the individuals without any partner or dependents from demographic information that is obtained during onboarding, and sending them a mini personalized survey as well, either via email or SMS/Whatsapp(if registered), asking whether they are satisfied so far and any concerning responses can be dealt with accordingly by reaching out and clarifying the issues being faced, so as to identifying potential customers who are at risk early, and take corrective measures.

5. Although **MonthlyCharges** has **high collinearity** with all other features, which is to be expected as it is a **direct result of them** (type of internet service, add-ons, etc), it has clearly been seen that with **tenure**, the **monthly charges increase**. However, it is also worthy of note that churned customers who were on the month-to-month contract type had significantly less monthly charge compared to the retained customers, whereas for the one- and two-year contract types, the average monthly charge for churned customers were higher than that of retained customers, signalling some kind of mismatch.

Suggestion:

- a. This shows the importance of regular feedback calls or mails or messages so that the company can keep understanding the customer's point of view to identify areas where the customers is suffering and also identify the areas where the customer is satisfied. This will help the company to reduce the customer dissatisfaction and in turn increase the chances of retention.
 - b. Special technical training can be provided to the employees of the company who onboard the customers, so that they can know the products provided by the company better and by understanding the products better, they can better understand the needs of the customer, which in turn will help them guide the customers towards plans and add-ons relevant to them, so that overspending can be avoided on the customer's side and satisfaction can be increased.
6. When it comes to PaperlessBilling and PaymentMethod, individuals who have paperless billing and paying via electronic check, have some of the highest churn. In fact, electronic check has some of the highest churn in itself. This could be due to several factors like electronic check payments being difficult, extra charges related to the payment, or, errors or difficulties being faced by the customers during payment. Individuals who are environmentally friendly and/or prefer to be organized, may opt for both paperless billing and electronic check payment. Since electronic check already has high churn, paperless billing might add to it if the bills are inconsistent, the billing is not done on time and/or bills are not sent to the respective places in time. This can increase the frustration among customers that will eventually lead to customers Churning.

Suggestions:

- a. Personalized feedback message sent via mail/SMS (or Whatsapp if registered), or a feedback page that pops up after making payment, that asks customers how their payment experience was and if it was seamless. An added requirement of a text section that further explains what the difficulty was will provide deeper insights into the problems being faced, so that they can be fixed quickly.
- b. Going over the entire billing system, automating as much of it as possible via machines that feed data into databases and agentic AI workflows that can create the bill structure and fill it accordingly, might lead to a reduction in human induced errors or inconsistencies in billing that might increase the satisfaction and reduce the chances of churn.
- c. Customers with high monthly charges and paying via electronic check are already at risk and apart from above mentioned measures, providing them with occasional discounts on charges and discounts on bundled services might bring down the chances of churn.

Immediate Recommendations

1. Send a feedback form via mail first to all retained customers asking on a scale of 1 to 5 how they would rate their experience or satisfaction regarding different services availed by them, followed by Whatsapp (if registered) or calls till we get the highest amount of response possible. A separate feedback call to be made to churned customers to identify key areas where they faced problems that ultimately made them churn.
2. Employees that onboard customers should be given training in batches to make sure that their knowledge about the products and services are adequate and up to date so that they will be better able to assist customers.
3. A specialized triage team should be assembled, comprised of employees with deep knowledge about the products and services, so that they might be able to analyse the feedback responses.
4. A special discount can be given to customers opting for fiber optic connections along with add-ons in the one- and two-year contract types.
5. The billing team must assess the payment procedure and identify the problems customers are facing in electronic check payments and those receiving paperless billing. A feedback section must appear after payment that lets the customer rate how easy it was to make the payment, and if any problems were faced.

Preparing the Data for Modelling

Addressing The Main Concern

The dataset showed a class imbalance in the target variable (Churn), which must be addressed while performing the train-test split, with K-fold cross validation on the training set, and later when modelling the data for the machine learning algorithms. Since the MonthlyCharges and TotalCharges were dropped as per the VIF finding, which were numerical features, leaving Tenure as the only numerical column, and the remaining features are all categorical. Taking this into consideration, the following approaches were selected for the resampling of the data after performing the train-test split and before performing the K-fold cross-validation.

- **Stratified random sampling** – This ensures that the representation of the categories in the training, and testing dataset is similar to the original, and the distribution of the original data is preserved.
- **SMOTENC** – This is an oversampling method whereby synthetic data is created, for the underrepresented category, so that there is improvement in the representation of the minority class in the target variable. This is being chosen over SMOTE because SMOTENC preserves categorical integrity while generating synthetic data.
- **SMOTEENN** – This is a hybrid technique that performs both oversampling and undersampling, by eliminating some of the noisy and borderline data from the majority class, and adding synthetic data for the undersampled category in the training dataset.
- **SMOTETomek** – Similar to SMOTEENN this is also a hybrid technique that performs both oversampling and undersampling, but unlike SMOTEENN, it is less harsh and will lead to more instances of the original data (belonging to the majority class) being preserved. This is important because given a dataset size of 7032 instances and a train-test split of 80-20 with K-fold cross validation, there might be removal of useful data along with the noisy and borderline data while using SMOTEENN.

Apart from the aforementioned, an initial round of logistic regression will be performed on the dataset with MonthlyCharges and TotalCharges preserved using L1 regularization to see if the model validates the finding that these features introduce unwanted redundancy, through feature selection induced by L1 regularization.

The Importance of Using K-Fold Cross Validation

In this project, the dataset is divided into train and test sets, and K-fold cross-validation is used on the training set, keeping in mind several key factors such as:

- Dataset Size (7032 instances)
- Class Imbalance (26.58% Churned vs 73.42% Non-churned)
- Bias in Model Selection

The last point is the most important as we will be evaluating different models and experimenting with different sets of hyperparameters to select the best-performing model. Doing this process only with a training and test set, without cross validation, will lead to bias as the model will be selected based on the performance on the test set. To avoid this, K-Fold Cross Validation is performed on the training set which divides it into K sets (K-1 training and 1 validation set) and rotates it K times, and in each rotation, the validation set is changed. This way, each sample is used for validation exactly once.

Data Preparation

The csv is loaded into a new Jupyter Notebook using Pandas and the first 5 rows are viewed.

```
import pandas as pd
df = pd.read_csv("telco_customer_churn_clean.csv")
df.head()
```

Initial dataset observation done in excel showed that there are a few missing and non-numeric values in the TotalCharges column. These are viewed and then removed.

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
df.dropna(inplace=True)
```

Now there are 7032 instances that can be worked with.

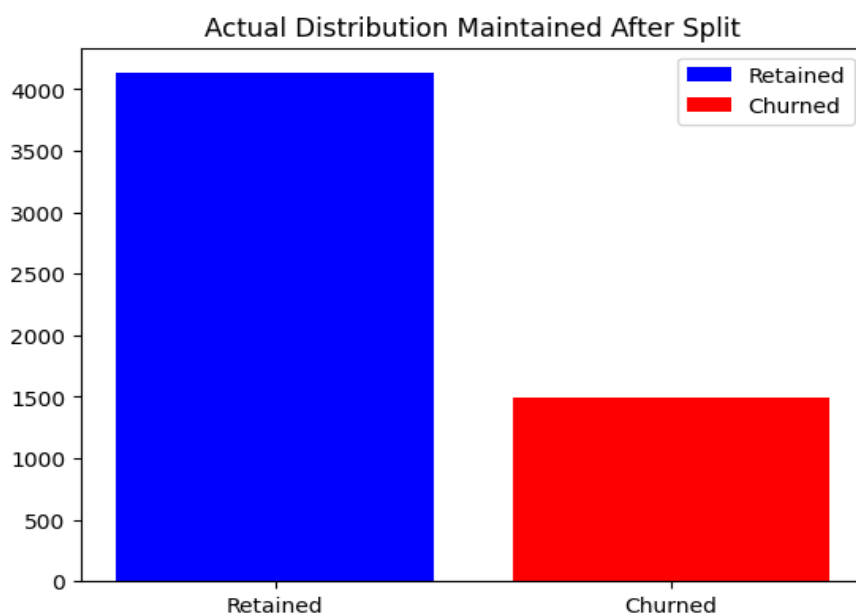
The first thing that is done now is to define the independent and dependent variables and then to split the data into training and testing sets. This is done using scikit-learn's `train_test_split()` with the parameter `stratify=y`.

```
from sklearn.model_selection import train_test_split
df.drop(columns=["customerID"], inplace=True)
X = df.drop(columns=["Churn"])
y = df["Churn"]
X_train_init, X_test, y_train_init, y_test = train_test_split(X, y,
test_size=0.2, stratify=y, random_state=42)
```

The `X_test` and `y_test` datasets are now set aside not to be looked at till the evaluation at the end of the entire process.

Checking the distribution of the classes in the target feature, it is seen that the original structure is maintained.

```
import matplotlib.pyplot as plt
plt.bar(data=y_train_init, x=y_train_init["Churn"].unique(),
height=y_train_init["Churn"].value_counts(),
color=["b", "r"], label=["Retained", "Churned"],
tick_label=["Retained", "Churned"])
plt.title("Actual Distribution Maintained After Split")
plt.legend()
plt.show()
```



Using scikit-learn's `OneHotEncoder()`, the nominal variables are encoded, using the `OrdinalEncoder()`, the ordinal variables are encoded, using the `LabelEncoder()`, the target variable is encoded and using `StandardScaler()`, the metric variables are encoded.

```
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder,
LabelEncoder, StandardScaler
# one hot encoder
ohe = OneHotEncoder(
    handle_unknown="error",
    sparse_output=False
).set_output(transform="pandas")
# ordinal encoder
oe = OrdinalEncoder(
    handle_unknown="error"
).set_output(transform="pandas")
# label encoder
le = LabelEncoder()
# standard scaler
ss = StandardScaler().set_output(transform="pandas")
```

The classes in multiclass features such as `OnlineBackup` and `TechSupport` were adjusted so that they become binary (Yes/No), since the feature `InternetService` already has "No internet service", etc. Also, creating two separate lists, for numerical features and categorical features.

```
# adjusting the classes in each feature
for col in X_train_init.columns.tolist():
    if col in ["OnlineSecurity", "OnlineBackup",
              "DeviceProtection", "TechSupport", "StreamingTV",
              "StreamingMovies"]:
        X_train_init[col] = X_train_init[col].apply(lambda x: "No" if
x=="No internet service" else x)
    elif col=="MultipleLines":
        X_train_init[col] = X_train_init[col].apply(lambda x: "No" if
x=="No phone service" else x)
# creating list to separate data types
categorical_columns = []
numerical_columns = []
# running a for loop to append the lists
for col in X_train_init.columns.tolist():
    if col=="SeniorCitizen":
        continue
    elif X_train_init[col].dtype=="str":
        categorical_columns.append(col)
    else:
        numerical_columns.append(col)
```

After final definition of the columns and how each should be encoded, the following loop is run so as to encode the data accordingly.

```
# columns to be one hot encoded
OneHot = ["InternetService", "Contract", "PaymentMethod"]

# columns to be ordinally encoded
# Note: Even though some columns here are nominal, they are binary, and can
be encoded as ordinal
Ordinal = [col for col in categorical_columns if col not in OneHot]

X_train = X_train_init[["SeniorCitizen"]]
```

```

for col in X_train_init.columns.tolist():
    if col in OneHot:
        interim = ohe.fit_transform(X_train_init[[col]])
        X_train = pd.concat([X_train, interim], axis=1)
    elif col in Ordinal:
        interim = oe.fit_transform(X_train_init[[col]])
        X_train = pd.concat([X_train, interim], axis=1)
    elif col in numerical_columns:
        interim = ss.fit_transform(X_train_init[[col]])
        X_train = pd.concat([X_train, interim], axis=1)
    elif col=="SeniorCitizen":
        X_train[col] = X_train[col].astype("float64")
    else:
        print(f"Error encoding column : {col}")

X_train.drop(columns=["InternetService_No", "Contract_Two year",
                      "PaymentMethod_Mailed check"], inplace=True)

y_train_interim = le.fit_transform(y_train_init)

y_train = pd.Series(
    data=y_train_interim
)

```

Next, the dataset, with MonthlyCharges and TotalCharges still existing, will be run through L1 regularized Linear Regression at separate values of regularization strength, to see the behaviour of coefficients.

```

from sklearn.linear_model import LogisticRegression
import warnings
warnings.filterwarnings("ignore")
C_vals = [0.01, 0.03, 0.09, 0.3, 0.9, 3, 9]
cof = {}
iteration = 1
for c in C_vals:
    print(f"C Value number {iteration} : C Value = {C_vals[iteration-1]}")
    feature_val = LogisticRegression(
        solver="liblinear",
        penalty="l1",
        C = c,
        l1_ratio=1.0
    )
    feature_val.fit(X_train, y_train)
    cof[c]=feature_val.coef_
    print(f"For C Value number {iteration}, training is complete")
    iteration += 1

import numpy as np
coeff_zeros = np.zeros((23,7))
coeff_df =
pd.DataFrame(coeff_zeros, index=X_train.columns.to_list(), columns=C_vals)

for i,j in cof.items():
    coeff_df[i] = j[0]

coeff_df.to_csv("coeff_before_drop.csv")
coeff_df

```

Running the above code, gave rise to the following table.

Features	0.01	0.03	0.09	0.3	0.9	3	9
PhoneService	-0.592303875	-0.922841798	-1.076739634	-1.17668036	-1.116936156	-0.659498338	0.098688475
tenure	-0.900506223	-0.842267066	-0.916309452	-1.22768251	-1.337836161	-1.383085323	-1.401475427
TechSupport	-0.338555023	-0.496556564	-0.536239587	-0.537438396	-0.514453793	-0.402241997	-0.21169904
OnlineSecurity	-0.297122266	-0.481198254	-0.540122796	-0.56275174	-0.550040493	-0.443104519	-0.254927436
Dependents	-0.065308334	-0.205015886	-0.238333899	-0.234429385	-0.232195523	-0.230919557	-0.229707008
OnlineBackup	0	-0.169983699	-0.259529668	-0.30986043	-0.305216763	-0.200542481	-0.014579902
DeviceProtection	0	-0.025799312	-0.093917811	-0.119680416	-0.104557967	0	0.191088808
InternetService_DSL	0	-0.020512392	-0.037353151	0	0.147816087	0.77338795	1.739770927
gender	0	0	0	-0.016465074	-0.023023848	-0.024306826	-0.022797749
Partner	0	0	-0.005274155	-0.017250989	-0.020504372	-0.022510652	-0.025146115
MultipleLines	0	0	0.071658956	0.131869363	0.170746643	0.289812045	0.481884278
InternetService_Fiber optic	0	0	0	0.072951599	0.337509635	1.529287347	3.446836064
StreamingTV	0	0	0	-0.018312917	0	0.226992695	0.606462982
StreamingMovies	0	0	0	-0.050399501	-0.018233893	0.197873799	0.578961926
Contract_One year	0	0	0	0.360876888	0.545656708	0.614326203	0.637954987
PaymentMethod_Bank transfer (automatic)	0	0	-0.016859321	-0.0450122	-0.061926791	-0.072259456	-0.076685701
PaymentMethod_Credit card (automatic)	0	0	0	-0.002854006	-0.019732298	-0.029334491	-0.032658496
TotalCharges	0	0	0.091735637	0.47914025	0.617914065	0.674767407	0.695077908
SeniorCitizen	0	0.014924679	0.137050092	0.176515525	0.187143119	0.190903842	0.192616132
PaperlessBilling	0	0.070157223	0.217399026	0.272740675	0.288935874	0.291382428	0.290307637
PaymentMethod_Electronic check	0.055187299	0.255917057	0.309187595	0.322928683	0.318040197	0.310800094	0.307640925
Contract_Month-to-month	0.082165368	0.608507224	0.813554736	1.158216726	1.324013794	1.381790624	1.402050487
MonthlyCharges	0.779576583	0.937656581	0.953865867	0.86149226	0.700469272	0	-1.150823572

Interpretation: It is clear from the trend shown by the coefficients above that having a low tenure, month-to-month contract, high charges, electronic check payment, and not having dependents, or online security, or phone service or tech support is associated with churn. But, due to the coefficient instability exhibited by coefficients across different regularization strengths for the L1 regularization, it is clear that multicollinearity exists.

The TotalCharges column will now be dropped to see the effect it has on multicollinearity. And the L1 Logistic Regression model will be run again to look for coefficient stabilization.

```
X_train.drop(columns="TotalCharges", inplace=True)
cof = {}
iteration = 1
for c in C_vals:
    print(f"C Value number {iteration} : C Value = {C_vals[iteration-1]}")
    feature_val = LogisticRegression(
        solver="liblinear",
        penalty="l1",
        C = c,
        l1_ratio=1.0
    )
    feature_val.fit(X_train,y_train)
    cof[c]=feature_val.coef_
    print(f"For C Value number {iteration}, training is complete")
    iteration += 1
coeff_zeros = np.zeros((22,7))
coeff_df =
pd.DataFrame(coeff_zeros,index=X_train.columns.to_list(),columns=C_vals)

for i,j in cof.items():
    coeff_df[i] = j[0]

coeff_df.to_csv("coeff_after_one_drop.csv")
coeff_df
```

Features	0.01	0.03	0.09	0.3	0.9	3	9
PhoneService	-0.589397354	-0.92298211	-1.073249026	-1.110872965	-1.129132958	-0.523032276	0.074748108
tenure	-0.900177669	-0.842366909	-0.839792639	-0.819219535	-0.810286014	-0.805474255	-0.805525624
TechSupport	-0.337816994	-0.496573802	-0.532751708	-0.513792653	-0.506816499	-0.356973215	-0.206742727
OnlineSecurity	-0.29657384	-0.481201599	-0.53781714	-0.543254471	-0.544703928	-0.400870066	-0.252498926
Dependents	-0.065108865	-0.205006504	-0.240128612	-0.243817691	-0.244532336	-0.244073179	-0.243419459
OnlineBackup	0	-0.169977911	-0.255226008	-0.279121492	-0.287182592	-0.144615985	0
DeviceProtection	0	-0.02583868	-0.0893314	-0.092383276	-0.093721378	0.047897421	0.199501036
InternetService_DSL	0	-0.020553148	-0.043797827	0	0.001652359	0.799751387	1.561065651
gender	0	0	0	-0.014507477	-0.020876977	-0.021669502	-0.020771617
Partner	0	0	-0.005960634	-0.019571724	-0.0238412	-0.026387441	-0.028480858
MultipleLines	0	0	0.074869057	0.157842162	0.181427203	0.338371437	0.490948852
InternetService_Fiber optic	0	0	0	0.133092396	0.159070184	1.706312207	3.218291098
StreamingTV	0	0	0	0	0	0.30116969	0.601082455
StreamingMovies	0	0	0	-0.009716888	-0.016718315	0.273798447	0.575035403
Contract_One year	0	0	0	0.353078344	0.530198384	0.595136016	0.616112351
PaymentMethod_Bank transfer (automatic)	0	0	-0.020856688	-0.081397439	-0.105667025	-0.119777544	-0.125099256
PaymentMethod_Credit card (automatic)	0	0	0	-0.037581818	-0.061278887	-0.074274088	-0.078573146
SeniorCitizen	0	0.014887715	0.136568432	0.175501799	0.186127004	0.190000266	0.191398038
PaperlessBilling	0	0.07013628	0.216442902	0.270525399	0.286479094	0.288134985	0.287432472
PaymentMethod_Electronic check	0.05563588	0.256030214	0.305972171	0.292023903	0.28137102	0.271200823	0.267303104
Contract_Month-to-month	0.083252291	0.60829732	0.81198592	1.14520424	1.304574485	1.357168951	1.374792318
MonthlyCharges	0.778979839	0.937690149	0.975504868	0.917504335	0.906879016	0	-0.904395488

Interpretation: The stabilization of the coefficients after removing TotalCharges, reveals that there was significant multicollinearity, most likely between MonthlyCharges, Tenure and TotalCharges. The coefficients are now significantly more stable, revealing that the data is now ready for the predictive algorithms.

Final Take

- Since removing TotalCharges stabilized the coefficients, dropping that feature will prove meaningful.
- Up until a significant regularization strength of 0.3, the coefficient of Gender is 0 and from the chi-square findings, it was clear that Gender is not associated with Churn, as such, the Gender feature will be removed.
- Chi-square had also revealed that PhoneService does not have any association with Churn and the coefficient of PhoneService had flipped at high regularization strength. As such, this feature will also be removed.
- Although MonthlyCharges also flipped sign at high regularization strength, across all other values, it steadily grew as the regularization strength grew, and it being an important business feature, this will be kept.
- The final features going ahead will be:
 - SeniorCitizen
 - Partner
 - Dependents
 - MultipleLines
 - InternetService
 - OnlineSecurity
 - OnlineBackup
 - DeviceProtection
 - TechSupport
 - StreamingTV
 - StreamingMovies
 - Contract
 - PaperlessBilling
 - PaymentMethod
 - Tenure
 - MonthlyCharges

Model Building and Performance Evaluation

Helper Function

Before starting the model building phase, some helper functions can be made to prepare the data more easily as was done in the last section, so that it can be automated. The following code performs all the previously mentioned tasks and returns the proper and encoded X_train, X_test, y_train and y_test.

```
import pandas as pd
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder,
LabelEncoder, StandardScaler

main_dir = Path(__file__).resolve().parent.parent.parent

notebook = main_dir / "notebooks"
src = main_dir / "src"
data = main_dir / "data"

def load_data(dataset:str):

    df = pd.read_csv(f"{data}/{dataset}")
    print("Dataset '{}' loaded successfully.\n")

    df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
    print("TotalCharges was typecasted to numerical.\n")

    df.dropna(inplace=True)
    print("Null values were removed.\n")
    print(f"\nDataset size : \n{df.shape[0]} rows\n{df.shape[1]}
columns\n\n")

    df.drop(columns=["customerID",
                    "gender",
                    "PhoneService",
                    "TotalCharges"], inplace=True)
    print("Successfully dropped the columns\n" \
          "-customerID\n" \
          "-gender\n" \
          "-PhoneService\n" \
          "-TotalCharges\n\n"
    )

    X = df.drop(columns=["Churn"])
    y = df["Churn"]
    X_train_init, X_test_init, y_train_init, y_test_init= train_test_split(
        X, y,
        test_size=0.2,
        stratify=y,
        random_state=42
    )

    print("Dataset split complete.")

    # one hot encoder
    ohe = OneHotEncoder(
        handle_unknown="error",
```

```

    sparse_output=False
    ).set_output(transform="pandas")

# ordinal encoder
oe = OrdinalEncoder(
    handle_unknown="error"
    ).set_output(transform="pandas")

# label encoder
le = LabelEncoder()

# standard scaler
ss = StandardScaler().set_output(transform="pandas")

# adjusting the classes in each feature
for col in X_train_init.columns.tolist():
    if col in ["OnlineSecurity", "OnlineBackup",
               "DeviceProtection", "TechSupport",
               "StreamingTV", "StreamingMovies"]:
        X_train_init[col] = X_train_init[col].apply(
            lambda x: "No" if x=="No internet service" else x
        )
        X_test_init[col] = X_test_init[col].apply(
            lambda x: "No" if x=="No internet service" else x
        )
    elif col=="MultipleLines":
        X_train_init[col] = X_train_init[col].apply(
            lambda x: "No" if x=="No phone service" else x
        )
        X_test_init[col] = X_test_init[col].apply(
            lambda x: "No" if x=="No phone service" else x
        )

# creating list to separate data types
categorical_columns = []
numerical_columns = []

# running a for loop to append the lists
for col in X_train_init.columns.tolist():
    if col=="SeniorCitizen":
        continue
    elif X_train_init[col].dtype=="str":
        categorical_columns.append(col)
    else:
        numerical_columns.append(col)

# columns to be one hot encoded
OneHot = ["InternetService", "Contract", "PaymentMethod"]

# columns to be ordinally encoded
# Note: Even though some columns here are nominal, they are binary,
and can be encoded as ordinal
Ordinal = [col for col in categorical_columns if col not in OneHot]

X_train = X_train_init[["SeniorCitizen"]]
X_test = X_test_init[["SeniorCitizen"]]

for col in X_train_init.columns.tolist():
    if col in OneHot:
        interim_train = ohe.fit_transform(X_train_init[[col]])
        X_train = pd.concat([X_train, interim_train], axis=1)

```

```

        interim_test = ohe.transform(X_test_init[[col]])
        X_test = pd.concat([X_test, interim_test], axis=1)
    elif col in Ordinal:
        interim_train = oe.fit_transform(X_train_init[[col]])
        X_train = pd.concat([X_train, interim_train], axis=1)
        interim_test = oe.transform(X_test_init[[col]])
        X_test = pd.concat([X_test, interim_test], axis=1)
    elif col in numerical_columns:
        interim_train = ss.fit_transform(X_train_init[[col]])
        X_train = pd.concat([X_train, interim_train], axis=1)
        interim_test = ss.transform(X_test_init[[col]])
        X_test = pd.concat([X_test, interim_test], axis=1)
    elif col=="SeniorCitizen":
        X_train[col] = X_train[col].astype("float64")
        X_test[col] = X_test[col].astype("float64")
    else:
        print(f"Error encoding column : {col}")

X_train.drop(columns=["InternetService_No", "Contract_Two year",
                    "PaymentMethod_Mailed check"],inplace=True)

X_test.drop(columns=["InternetService_No", "Contract_Two year",
                    "PaymentMethod_Mailed check"],inplace=True)

y_train_interim = le.fit_transform(y_train_init)
y_test_interim = le.transform(y_test_init)

y_train = pd.Series(
    data=y_train_interim,
    index=y_train_init.index
)

y_test = pd.Series(
    data=y_test_interim,
    index=y_test_init.index
)

return X_train, X_test, y_train, y_test

```

With the above helper function, the data can now be easily loaded, transformed, encoded and split accordingly, after which the model training and evaluation phase can begin.

Models under consideration

1. Linear Models
 - a. Logistic regression without any regularization to get a baseline performance
 - b. Logistic regression with L2 regularization without any over- or under-sampling
 - c. Logistic regression with L2 regularization and data oversampling with SMOTENC
 - d. Logistic regression with L2 regularization and sampling done via SMOTEENN
 - e. Logistic regression with L2 regularization and sampling done via SMOTETomek
2. Tree Based Models
 - a. Basic decision tree to get a baseline performance
 - b. RandomForest
 - c. XGBoost
 - d. CatBoost
 - e. LightGBM

Linear Models

Logistic Regression Without any Regularization

For this, all the necessary libraries were imported and then the training was carried out using K-fold cross validation with a cv value of 5.

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_validate
from pathlib import Path
import sys
base_dir = Path().resolve().parent
sys.path.append(str(base_dir / "src"))
from helpers.load_data import load_data
import warnings
warnings.filterwarnings("ignore")

dataset = "telco_customer_churn_clean.csv"
X_train, X_test, y_train, y_test = load_data(dataset)

modell1 = LogisticRegression()

skf = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

metrics = ["accuracy", "precision", "recall", "f1"]

scores_model1 = cross_validate(
    estimator=modell1,
    X=X_train,
    y=y_train,
    cv=skf,
    scoring=metrics
)

scores_model2 = {}

df_dict = {}
for i,j in scores_model1.items():
    df_dict[i]=j.mean()
lr1_df = pd.DataFrame(data=df_dict.values(), index=df_dict.keys(),
columns=["Mean_Score"])
lr1_df.to_csv(base_dir / "data" / "log_reg_no_regu.csv")
```


The above code gave rise to the following table which shows the metrics obtained by the baseline model.

Metric	Mean_Score
fit_time	0.052809429
score_time	0.027741957
test_accuracy	0.8
test_precision	0.650933104
test_recall	0.535117057
test_f1	0.587104918

Interpretation: The above baseline model has a precision of 0.65 which implies that 65% of all customers who were predicted to churn, actually churned. But here the recall is only 0.53. This implies that only 53% of all churners were identified leaving the other 47% who churned. Using the argument stated at the beginning, if there were 2000 customers at risk of churning, only 53% of that were currently identified, leaving the other 47% who will finally churn. That is a total of ~940 customers, which would lead to a revenue loss of roughly \$56,400 per month @ \$60 average monthly charge.

Logistic Regression with L2 Regularization at Different Regularization Strengths

For this, the previously split dataset was used, and the model was trained on different regularization strengths of 0.01, 0.03, 0.09, 0.3, 0.9, and 3.

```
scores_model2 = {}
c = [0.01, 0.03, 0.09, 0.3, 0.9, 3]

for c_val in c:
    model2 = LogisticRegression(
        solver='liblinear',
        penalty='l2',
        C=c_val,
        l1_ratio=0
    )

    scores_model = cross_validate(
        estimator=model2,
        X=X_train,
        y=y_train,
        cv=skf,
        scoring=metrics
    )

    scores_model2[c_val] = scores_model

zeros = np.zeros((4,6))
lr2_df =
pd.DataFrame(data=zeros, index=['test_accuracy', 'test_precision', 'test_recall', 'test_f1'], columns=c)
for c_val in c:
    curr = scores_model2[c_val]
    for metric in metrics:
        mean_score = curr[f"test_{metric}"].mean()
        lr2_df.loc[f"test_{metric}", c_val] = mean_score
```

```
lr2_df.to_csv(base_dir / "data" / "log_reg_l2_regu_diff_c.csv")
```

The code above gave rise to the following table:

Metrics	0.01	0.03	0.09	0.3	0.9	3
test_accuracy	0.795022222	0.796444444	0.797155556	0.801066667	0.800533333	0.799644444
test_precision	0.656186477	0.652105714	0.649924388	0.656689823	0.652901071	0.649954246
test_recall	0.480936455	0.502341137	0.515050167	0.528428094	0.533779264	0.535117057
test_f1	0.554660812	0.567118425	0.574215318	0.585119211	0.587088408	0.586704712

Interpretation: With regularization, the model performed comparatively worse than the model without regularization and was able to reach the performance of the model without regularization at a relatively high value of C. Since in scikit-learn's LinearRegression() module,

$$C = \frac{1}{\lambda}$$

a higher C means a lower regularization strength. At a C value of 3, the model had almost identical metric scores as the baseline model, with 0.53 as recall and an f1 score of 0.58.

Why Use Recall and F1 Score Instead of Accuracy

While accuracy shows the overall performance of the model, in this business context, a missed churner can be more expensive than a false positive where a non-churner is predicted to churn. This is in line with the fact that a missed churner leads to reduction in CLTV and overall revenue. It also means that the CAC will grow as well, as a certain number of customers are necessary to keep revenue growing, or at least stable, and not declining. This is highly relevant in this dataset as it has an inherent class imbalance whereby 26.58% of the data is of people who churned.

Logistic Regression With and Without Regularization Using SMOTENC

As a next step, SMOTENC and StratifiedKFold were used to oversample the minority class in the target variable per fold, keeping each cross validation set untouched. First a simple linear regression model was trained on it without any regularization and then an L2 regularized linear regression model was trained on it using separate values of regularization strength.

```
import numpy as np
import pandas as pd
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.linear_model import LogisticRegression
from imblearn.over_sampling import SMOTENC
from imblearn.combine import SMOTEENN, SMOTETomek
from imblearn.pipeline import Pipeline
from pathlib import Path
import sys
base_dir = Path().resolve().parent
sys.path.append(str(base_dir / "src"))
from helpers.load_data import load_data
import warnings
warnings.filterwarnings("ignore")

dataset = "telco_customer_churn_clean.csv"
```

```

X_train, X_test, y_train, y_test = load_data(dataset)

model = LogisticRegression()

skf = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

cat_cols = ["SeniorCitizen", "Partner", "Dependents",
            "MultipleLines", "InternetService_DSL", "InternetService_Fiber
            optic",
            "OnlineSecurity", "OnlineBackup", "DeviceProtection",
            "TechSupport", "StreamingTV", "StreamingMovies",
            "Contract_Month-to-month", "Contract_One year",
            "PaperlessBilling", "PaymentMethod_Bank transfer (automatic)",
            "PaymentMethod_Credit card (automatic)",
            "PaymentMethod_Electronic check"]

smotenc = SMOTENC(
    categorical_features=cat_cols,
    random_state=42
)

pipeline = Pipeline([
    ("smote", smotenc),
    ("model", model)
])

metrics = ["accuracy", "precision", "recall", "f1"]

scores_model = cross_validate(
    estimator=pipeline,
    X=X_train,
    y=y_train,
    cv=skf,
    scoring=metrics
)

temp_arr = np.zeros((6,1))
temp_df =
pd.DataFrame(temp_arr, index=scores_model.keys(), columns=["Mean_Score"])
for metric in metrics:
    mean_score = scores_model[f"test_{metric}"].mean()
    if f"test_{metric}" in scores_model.keys():
        temp_df.loc[metric, "Mean_Score"] = mean_score
    print(f"Metric : {metric} = Mean Score : {mean_score}")
temp_df.to_csv(base_dir / "data" / "smotenc_log_reg_NoRegu.csv")

```

The code above gave rise to the following table

Metric	Mean_Score
test_accuracy	0.755733333
test_precision	0.527483901
test_recall	0.775250836
test_f1	0.627725729

We can see here that recall here is much better compared to the simple linear regression model, where no oversampling was done. But, this has caused the precision to go down. For this model, the recall is at ~ 0.78 and the precision is about ~ 0.53 . This implies that 78% of churners will be correctly identified before they can churn. This means that if 2000 people would have churned, the model would be able to identify ~ 1560 of them saving almost \$93,600 in monthly revenue, if the average monthly charge per customer was @60. But the precision here is 0.53 which means that 47% of all those who would be identified as a churn risk, would be false positives. Although, a less of an expense compared to the loss in CLTV and CAC, this still leads to unwanted expenses and time consumption resulting from retention efforts that might take place such as discounted prices, offers, and feedback, towards customers who would not have churned anyway.

The following code now runs an L2 regularized logistic regression models and trains it using SMOTENC sampled data and StratifiedKFold cross validation, across different regularization strengths.

```
c = [0.01, 0.03, 0.09, 0.3, 0.9, 3]

scores = {}

for c_val in c:
    skf = StratifiedKFold(
        n_splits=5,
        shuffle=True,
        random_state=42
    )
    model = LogisticRegression(
        solver="liblinear",
        penalty="l2",
        C=c_val,
    )
    smotenc = SMOTENC(
        categorical_features=cat_cols,
        random_state=42
    )
    pipeline = Pipeline([
        ("smote", smotenc),
        ("model", model)
    ])
    scores_model = cross_validate(
        estimator=pipeline,
        scoring=metrics,
        cv=skf,
        X=X_train,
        y=y_train
    )
    scores[c_val]=scores_model
new_arr = np.zeros((len(metrics), len(c)))
new_df = pd.DataFrame(new_arr, index=metrics, columns=c)
for c_val in c:
    curr_score = scores[c_val]
    print(f"For C value of {c_val}")
    for metric in metrics:
        mean_score = curr_score[f"test_{metric}"].mean()
        new_df.loc[metric, c_val]=mean_score
        print(f"Metric : {metric} = Mean_score : {mean_score}")
    print()

new_df.to_csv(base_dir / "data" / "smotenc_lin_reg_l2_regu_DiffC.csv")
```

The code above gave rise to the table below:

Metrics	0.01	0.03	0.09	0.3	0.9	3
accuracy	0.750222222	0.756266667	0.756444444	0.755555556	0.755377778	0.755733333
precision	0.52047641	0.528536358	0.528907109	0.527556081	0.527071968	0.52751265
recall	0.772575251	0.767892977	0.767892977	0.770568562	0.773913043	0.77458194
f1	0.621770427	0.626064593	0.626273111	0.626199522	0.626999335	0.627534854

Interpretation: From the table above it is clear that the precision and recall scores are similar across different values of C. This implies that the model was not overfitting the data at all, which led to the scores being consistent, which was expected since the data was cleaned and scaled, multicollinearity addressed and redundant features removed.

Logistic Regression With and Without Regularization Using SMOTEENN

SMOTEENN and StratifiedKFold were used for combined sampling the minority class, using the edited nearest neighbours, in the target variable per fold, keeping each cross validation set untouched. First a simple linear regression model was trained on it without any regularization and then an L2 regularized linear regression model was trained on it using separate values of regularization strength.

```
model = LogisticRegression()

skf = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=45
)

smoteenn = SMOTEENN(random_state=42)

pipeline = Pipeline([
    ("smote", smoteenn),
    ("model", model)
])

model_scores = cross_validate(
    estimator=pipeline,
    X=X_train,
    y=y_train,
    cv=skf,
    scoring=metrics
)

new_arr = np.zeros((4,1))
new_df = pd.DataFrame(new_arr, index=metrics, columns=["Mean_Score"])
for metric in metrics:
    mean_score = model_scores[f"test_{metric}"].mean()
    new_df.loc[f"test_{metric}", "Mean_Score"] = mean_score
    print(f"Metric : {metric} = Mean_Score : {mean_score}")

new_df.to_csv(base_dir / "data" / "smoteenn_log_reg_NoRegu.csv")
```

The code above gave rise to the following table:

Metrics	Mean_Score
test_accuracy	0.71182222
test_precision	0.476987873
test_recall	0.859531773
test_f1	0.612999735

Interpretation: The table above shows a much-improved recall, at 0.85 which implies that 85% of customers at risk of churning would be correctly identified. Taking the example used before, if there were 2000 actual customers at risk of churning, 1700 of them would be correctly identified, which would save a revenue of \$102,000 per month from being lost, at \$60 average monthly charge per customer. But with a precision of 0.47, 53% of the churners who were identified, had no risk of churn, which would mean increased cost and time will be wasted on retention efforts towards customers who would not have churned at all.

The following code now runs an L2 regularized logistic regression models and trains it using SMOTEENN sampled data and StratifiedKFold cross validation, across different regularization strengths.

```
c = [0.001, 0.003, 0.009, 0.01, 0.03, 0.09, 0.3, 0.9, 3]

scores = {}

for c_val in c:
    skf = StratifiedKFold(
        n_splits=5,
        shuffle=True,
        random_state=42
    )
    model = LogisticRegression(
        solver="liblinear",
        penalty="l2",
        C=c_val,
    )
    smoteenn = SMOTEENN(
        random_state=42
    )
    pipeline = Pipeline([
        ("smote", smoteenn),
        ("model", model)
    ])
    scores_model = cross_validate(
        estimator=pipeline,
        scoring=metrics,
        cv=skf,
        X=X_train,
        y=y_train
    )
    scores[c_val]=scores_model

new_arr = np.zeros((len(metrics), len(c)))
new_df = pd.DataFrame(new_arr, index=metrics, columns=c)

for c_val in c:
    curr_score = scores[c_val]
    print(f"For C value of {c_val}")
    for metric in metrics:
        mean_score = curr_score[f"test_{metric}"].mean()
        new_df.loc[metric, c_val]=mean_score
```

```

        print(f"Metric : {metric} = Mean_score : {mean_score}")
    print()

new_df.to_csv(base_dir / "data" / "smoteenn_lin_reg_l2_regu_DiffC.csv")

```

The code above gave rise to the following table:

Metrics	0.001	0.003	0.009	0.01	0.03	0.09	0.3	0.9	3
accuracy	0.656888889	0.677511111	0.689244444	0.690133333	0.698488889	0.705777778	0.708088889	0.708444444	0.708444444
precision	0.430724386	0.44626427	0.455873491	0.45659608	0.463934506	0.470608235	0.472878847	0.473183647	0.473237991
recall	0.904347826	0.885618729	0.873578595	0.872240803	0.862876254	0.856856187	0.85819398	0.857525084	0.858862876
f1	0.583514754	0.593461605	0.599059389	0.599370598	0.603314724	0.607431124	0.609722287	0.609819921	0.610215184

Interpretation: The table above shows that at a very strong regularization strength with a C value of 0.001, the recall is the best at 0.90 but the precision is worst at 0.43 with an f1-score of 0.58.

Logistic Regression With and Without Regularization Using SMOTETomek

A combined sampling technique of SMOTETomek was used on the training dataset along with StratifiedKFold cross validation to train a simple logistic regression model on the training data.

```

model = LogisticRegression()

sfk = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

smotetomek = SMOTETomek(random_state=42)

pipeline = Pipeline([
    ("smote", smotetomek),
    ("model", model)
])

model_scores = cross_validate(
    estimator=pipeline,
    X=X_train,
    y=y_train,
    cv=sfk,
    scoring=metrics
)

new_arr = np.zeros((4,1))
new_df = pd.DataFrame(new_arr, index=metrics, columns=["Mean_Score"])

for metric in metrics:
    mean_score = model_scores[f"test_{metric}"].mean()
    new_df.loc[metric, "Mean_Score"] = mean_score
    print(f"Metric : {metric} = Mean Score : {mean_score}")

new_df.to_csv(base_dir / "data" / "smotetomek_lin_reg_NoRegu.csv")

```

The code above gave rise to the following table:

Metric	Mean_Score
accuracy	0.750933333
precision	0.520915499
recall	0.785953177
f1	0.626481202

Interpretation: The table above shows a recall that is somewhat degraded compared to the SMOTEENN technique, but the precision here is comparatively better. With a recall of ~0.79, it will identify correctly 78% of total churners, which for 2000 actual churners implies correctly identifying ~1580 customers, which can save a revenue of \$94,800. With a precision of 0.52, 48% of total churners identified will be false positives, where money and time will be spent in retention efforts, as these customers were never at risk of churning.

With L2 regularization, using the same above mentioned constraints.

```
c = [0.001, 0.003, 0.01, 0.03, 0.09, 0.3, 0.9, 3]

scores = {}

for c_val in c:
    skf = StratifiedKFold(
        n_splits=5,
        shuffle=True,
        random_state=42
    )
    model = LogisticRegression(
        solver="liblinear",
        penalty="l2",
        C=c_val
    )
    smotetomek = SMOTETomek(
        random_state=42
    )
    pipeline = Pipeline([
        ("smote", smotetomek),
        ("model", model)
    ])
    scores_model = cross_validate(
        estimator=pipeline,
        scoring=metrics,
        cv=skf,
        X=X_train,
        y=y_train
    )
    scores[c_val] = scores_model

new_arr = np.zeros((len(metrics), len(c)))
new_df = pd.DataFrame(new_arr, index=metrics, columns=c)

for c_val in c:
    curr_score = scores[c_val]
    print(f"For C value of {c_val}")
    for metric in metrics:
```



```

mean_score = curr_score[f"test_{metric}"].mean()
new_df.loc[metric,c_val]=mean_score
print(f"Metric : {metric} = Mean_score : {mean_score}")
print()

new_df.to_csv(base_dir / "data" / "smoteenn_lin_reg_l2_regu_DiffC.csv")

```

The code above gave rise to the following table:

Metrics	0.001	0.003	0.01	0.03	0.09	0.3	0.9	3
accuracy	0.723377778	0.733688889	0.744533333	0.750755556	0.751822222	0.751111111	0.751111111	0.750222222
precision	0.488152501	0.499653263	0.512844921	0.520747889	0.522064825	0.521138451	0.521101012	0.519982167
recall	0.834113712	0.810702341	0.789297659	0.77993311	0.785284281	0.785284281	0.785953177	0.786622074
f1	0.615840909	0.618152713	0.621535329	0.624416834	0.627095807	0.626425147	0.626633317	0.626023324

Interpretation: With the SMOTETomek technique applied, using StratifiedKFold and training an L2 regularized LinearRegression model, it can be seen that for a C value of 0.09, the recall is ~0.79 implying 79% of all churners correctly identified and a precision of 0.52 implying that among all customers identified as a churn risk, 48% of them are false positives.

Comparison of Linear Models

Model	Recall	Precision	F1	Accuracy
SMOTETomek Logistic Regression	0.79	0.52	0.63	0.75
SMOTETomek L2 Regularized Linear Regression (C = 0.9)	0.79	0.52	0.63	0.75
SMOTENC Logistic Regression	0.78	0.53	0.63	0.76
SMOTENC L2 Regularized Logistic Regression (C = 0.9)	0.77	0.52	0.63	0.76
SMOTEENN Logistic Regression	0.86	0.48	0.61	0.71
SMOTEENN L2 Regularized Logistic Regression (C = 0.9)	0.86	0.47	0.6	0.71
Basic Logistic Regression	0.54	0.65	0.59	0.8
L2 Regularized Logistic Regression (C = 0.9)	0.53	0.65	0.59	0.8

From the table above, it can be observed that among the linear models, the non-regularized logistic regression model that has undergone SMOTETomek sampling along with StratifiedKFold cross validation, has had the best performance among all other models in terms of recall, precision and f1 score. Although the model trained with the SMOTENC sampling technique has marginally higher precision, it also has marginally lower recall and in this business case, it is cheaper to give a customer discounts and retain him/her, rather than lose a customer to churn as that will lead to eventual overall revenue loss in the form of declining CLTV and increasing CAC per month.

Tree Based Models

Helper Function

Since tree-based models, work differently than linear models, the encoding of the data will be different. Also, since tree based models perform splits based on the actual data, apart from the customerID column, which is a unique identifier, no other columns are dropped. The following helper functions loads the data, cleans it, performs the encoding necessary and then returns the training and testing datasets.

```
import numpy as np
import pandas as pd
from pathlib import Path
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder,
LabelEncoder
from sklearn.model_selection import train_test_split

base_dir = Path(__file__).resolve().parent.parent.parent

dataset = base_dir / "data" / "telco_customer_churn_clean.csv"

def tree_data(dataset):

    df = pd.read_csv(dataset)
    print(f"Dataset '{dataset}' loaded successfully.\n")

    df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
    print("TotalCharges was typecasted to numerical.\n")

    df.dropna(inplace=True)
    print("Null values were removed.\n")
    print(f"\nDataset size : \n{df.shape[0]} rows\n{df.shape[1]}
columns\n\n")

    df.drop(columns=["customerID"], inplace=True)
    print("Successfully dropped the column: 'customerID'")

    X = df.drop(columns=["Churn"])
    y = df["Churn"]

    X_train_init, X_test_init, y_train_init, y_test_init =
train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

    print("Dataset split complete.")

    # one hot encoder
    ohe = OneHotEncoder(
        handle_unknown="ignore",
        sparse_output=False
    ).set_output(transform="pandas")

    # ordinal encoder
    oe = OrdinalEncoder(
        handle_unknown="error"
```

```

    ).set_output(transform="pandas")

# label encoder
le = LabelEncoder()

# adjusting the classes in each feature
for col in X_train_init.columns.tolist():
    if col in ["OnlineSecurity", "OnlineBackup",
               "DeviceProtection", "TechSupport",
               "StreamingTV", "StreamingMovies"]:
        X_train_init[col] = X_train_init[col].apply(
            lambda x: "No" if x=="No internet service" else x
        )
        X_test_init[col] = X_test_init[col].apply(
            lambda x: "No" if x=="No internet service" else x
        )
    elif col=="MultipleLines":
        X_train_init[col] = X_train_init[col].apply(
            lambda x: "No" if x=="No phone service" else x
        )
        X_test_init[col] = X_test_init[col].apply(
            lambda x: "No" if x=="No phone service" else x
        )

# creating list to separate data types
categorical_columns = []
numerical_columns = []

# running a for loop to append the lists
for col in X_train_init.columns.tolist():
    if col=="SeniorCitizen":
        continue
    elif X_train_init[col].dtype=="str":
        categorical_columns.append(col)
    else:
        numerical_columns.append(col)

# columns to be one hot encoded
OneHot = ["InternetService", "Contract", "PaymentMethod"]

# columns to be ordinally encoded
# Note: Even though some columns here are nominal, they are
# binary, and can be encoded as ordinal
Ordinal = [col for col in categorical_columns if col not in OneHot]

X_train = X_train_init[["SeniorCitizen"]]
X_test = X_test_init[["SeniorCitizen"]]

for col in X_train_init.columns.tolist():
    if col in OneHot:
        interim_train = ohe.fit_transform(X_train_init[[col]])
        X_train = pd.concat([X_train, interim_train], axis=1)
        interim_test = ohe.transform(X_test_init[[col]])
        X_test = pd.concat([X_test, interim_test], axis=1)
    elif col in Ordinal:
        interim_train = oe.fit_transform(X_train_init[[col]])
        X_train = pd.concat([X_train, interim_train], axis=1)
        interim_test = oe.transform(X_test_init[[col]])
        X_test = pd.concat([X_test, interim_test], axis=1)
    elif col in numerical_columns:
        interim_train = X_train_init[[col]]

```

```

        X_train = pd.concat([X_train, interim_train], axis=1)
        interim_test = X_test_init[[col]]
        X_test = pd.concat([X_test, interim_test], axis=1)
    elif col=="SeniorCitizen":
        X_train[col] = X_train[col].astype("float64")
        X_test[col] = X_test[col].astype("float64")
    else:
        print(f"Error encoding column : {col}")

print("\nSuccessfully encoded all columns\n")

y_train_interim = le.fit_transform(y_train_init)
y_test_interim = le.transform(y_test_init)

y_train = pd.Series(
    data=y_train_interim,
    index=y_train_init.index
)

y_test = pd.Series(
    data=y_test_interim,
    index=y_test_init.index
)

print("\nTraining and testing datasets are ready to be used.\n")

return X_train, X_test, y_train, y_test

```

Basic Decision Tree

A simple decision tree was trained on the dataset obtained from the above preprocessing using different values of parameters as shown in the code below.

```

import pandas as pd
import numpy as np
from sklearn.model_selection import StratifiedKFold, cross_validate
from imblearn.over_sampling import SMOTENC
from imblearn.combine import SMOTEENN, SMOTETomek
from imblearn.pipeline import Pipeline
from pathlib import Path
import sys
base_dir = Path().resolve().parent
sys.path.append(str(base_dir / "src"))
from helpers.tree_data import tree_data
import warnings
warnings.filterwarnings("ignore")
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier

dataset = base_dir / "data" / "telco_customer_churn_clean.csv"
X_train, X_test, y_train, y_test = tree_data(dataset)

max_depths = [3, 5, 7, 10, None]
min_samples_leafs = [1, 2, 5, 10, 20]
metrics = ["recall", "precision", "f1", "accuracy"]

new_arr = np.zeros((20,5))

```

```
new_df = pd.DataFrame(  
    new_arr,  
    index=pd.MultiIndex.from_product(  
        [max_depths, metrics],  
        names=["max_depth", "metric"]  
    ),  
    columns=min_samples_leafs  
)  
  
scores = {}  
  
for max_depth in max_depths:  
    for min_samples_leaf in min_samples_leafs:  
        model = DecisionTreeClassifier(  
            max_depth = max_depth,  
            min_samples_leaf = min_samples_leaf,  
            criterion = "gini",  
            class_weight = None,  
            random_state = 42  
        )  
  
        skf = StratifiedKFold(  
            n_splits = 5,  
            shuffle = True,  
            random_state = 42  
        )  
  
        model_scores = cross_validate(  
            estimator = model,  
            scoring = metrics,  
            X = X_train,  
            y = y_train,  
            cv = skf  
        )  
  
        scores[(max_depth,min_samples_leaf)] = model_scores  
  
for max_depth in max_depths:  
    for min_samples_leaf in min_samples_leafs:  
        print(f"Max Depth : {max_depth} | min_samples_leaf :  
{min_samples_leaf}")  
        curr_dict = scores[(max_depth,min_samples_leaf)]  
        for metric in metrics:  
            mean_score = curr_dict[f"test_{metric}"].mean()  
            print(f"Metric : {metric} | Mean Score : {mean_score}")  
            new_df.loc[(max_depth,metric),min_samples_leaf] = mean_score  
        print()  
  
new_df.to_csv(base_dir / "data" / "DT_Baseline_DiffParams.csv")
```

The code above gave rise to the following table, where the columns are the number of splits being performed on the leaf node and the indexes are the maximum depth.

max_depth	min_samples_leafs	recall	precision	f1	accuracy
5	10	0.55	0.60	0.57	0.78
5	20	0.54	0.60	0.57	0.78
5	1	0.54	0.60	0.57	0.78
5	2	0.54	0.60	0.57	0.78
5	5	0.54	0.60	0.57	0.78
7	20	0.52	0.62	0.57	0.79
7	1	0.52	0.60	0.55	0.78
7	10	0.52	0.61	0.56	0.78
7	2	0.52	0.60	0.55	0.78
7	5	0.52	0.59	0.55	0.78
10	1	0.51	0.55	0.53	0.76
10	5	0.51	0.56	0.53	0.76
10	10	0.50	0.59	0.54	0.78
10	2	0.50	0.57	0.53	0.77
10	20	0.50	0.62	0.55	0.79
None	20	0.49	0.63	0.55	0.79
None	1	0.49	0.49	0.49	0.73
None	10	0.49	0.59	0.53	0.77
None	5	0.47	0.54	0.50	0.75
None	2	0.45	0.51	0.48	0.74
3	1	0.36	0.70	0.48	0.79
3	2	0.36	0.70	0.48	0.79
3	5	0.36	0.70	0.48	0.79
3	10	0.36	0.70	0.48	0.79
3	20	0.36	0.70	0.48	0.79

From the table above, we can see that across different values for the maximum depth and the minimum samples in the leaf node, the recall, precision and f1 scores are much lower than the optimized linear models. In fact, the highest recall attained was only 0.55.

Of note, even though the train test split in the helper function used the parameter ``stratify=y`` for stratified sampling, the ``DecisionTreeClassifier`` from sklearn has a parameter called class_weight, which will now be set to "balanced" to further handle the imbalance in the dataset.

```
max_depths = [3, 5, 7, 10, None]
min_samples_leafs = [1, 2, 5, 10, 20]
metrics = ["recall", "precision", "f1", "accuracy"]

new_arr = np.zeros((25,4))
new_df = pd.DataFrame(
    new_arr,
    index=pd.MultiIndex.from_product(
        [max_depths, min_samples_leafs],
        names=["max_depth", "min_samples_leafs"]
    ),
    columns=metrics
)
```

```
scores = {}

for max_depth in max_depths:
    for min_samples_leaf in min_samples_leafs:
        model = DecisionTreeClassifier(
            max_depth = max_depth,
            min_samples_leaf = min_samples_leaf,
            criterion = "gini",
            class_weight = "balanced",
            random_state = 42
        )

        skf = StratifiedKFold(
            n_splits = 5,
            shuffle = True,
            random_state = 42
        )

        model_scores = cross_validate(
            estimator = model,
            scoring = metrics,
            X = X_train,
            y = y_train,
            cv = skf
        )

        scores[(max_depth,min_samples_leaf)] = model_scores

for max_depth in max_depths:
    for min_samples_leaf in min_samples_leafs:
        print(f"Max Depth : {max_depth} | min_samples_leaf :
{min_samples_leaf}")
        curr_dict = scores[(max_depth,min_samples_leaf)]
        for metric in metrics:
            mean_score = curr_dict[f"test_{metric}"].mean()
            print(f"Metric : {metric} | Mean Score : {mean_score}")
            new_df.loc[(max_depth,min_samples_leaf),metric] = mean_score
        print()

new_df.to_csv(base_dir / "data" / "DT_Baseline_DiffParams_balanced.csv")
```

The code above, gave rise to the following table:

max_depth	min_samples_leafs	recall	precision	f1	accuracy
5	20	0.80	0.51	0.62	0.74
5	10	0.80	0.50	0.62	0.74
5	5	0.79	0.50	0.62	0.74
5	1	0.79	0.50	0.61	0.74
5	2	0.79	0.50	0.61	0.74
7	20	0.79	0.50	0.61	0.73
7	5	0.79	0.49	0.61	0.73
7	10	0.78	0.49	0.60	0.72
7	2	0.78	0.49	0.60	0.72
3	1	0.78	0.52	0.62	0.75
3	2	0.78	0.52	0.62	0.75
3	5	0.78	0.52	0.62	0.75
3	10	0.78	0.52	0.62	0.75
3	20	0.78	0.52	0.62	0.75
7	1	0.78	0.49	0.60	0.73
10	20	0.77	0.49	0.60	0.73
None	20	0.77	0.50	0.60	0.73
10	10	0.74	0.48	0.58	0.72
10	5	0.73	0.49	0.59	0.72
None	10	0.70	0.47	0.57	0.71
10	2	0.70	0.47	0.56	0.71
10	1	0.69	0.48	0.57	0.72
None	5	0.67	0.49	0.57	0.72
None	2	0.57	0.47	0.52	0.71
None	1	0.49	0.50	0.49	0.73

The table above clearly shows that using `class_weight=balanced` in the `DecisionTreeClassifier` assignment along with a maximum tree depth of 5 and with 20 minimum samples on the leaf node, worked well, as now we have an impressive recall of 0.80 meaning 80% of all churners were identified which implies that in a business where there would have been potentially 2000 churners, 1600 of them would be correctly identified, which can realistically save the company from a revenue loss of ~\$96,000 @ \$60 average monthly charge per customer, and with a precision of 0.51, there would be a staggering 49% of all identified churners who would be false positives, which although would carry some cost in the form of customer retention efforts, would be much less compared to the revenue loss from losing customers, and CAC.

After trying the decision tree with balanced class weights, SMOTETomek and SMOTENC sampling methods, the score was not significantly improved and the best recall was attained by SMOTENC with a balanced class weight and the recall was ~0.67 implying that SMOTE sampling techniques will not be useful while using trees.

Random Forest

Then a random forest classifier model was trained on the dataset using balanced class weights.

```
n_estimators = [50, 100, 200, 300]
max_depths = [3, 5, 7, 10]
min_samples_leafs = [2, 5, 10, 20]
metrics = ["recall", "precision", "f1", "accuracy"]

new_arr = np.zeros((64,4))
new_df = pd.DataFrame(
    new_arr,
    columns=metrics,
    index=pd.MultiIndex.from_product(
        [n_estimators,max_depths,min_samples_leafs],
        names=["n_estimators","max_depths","min_samples_leafs"]
    )
)

scores = {}

for n_estimator in n_estimators:
    for max_depth in max_depths:
        for min_samples_leaf in min_samples_leafs:

            model = RandomForestClassifier(
                n_estimators=n_estimator,
                max_depth=max_depth,
                min_samples_leaf=min_samples_leaf,
                criterion="gini",
                class_weight="balanced",
                max_features="sqrt"
            )

            skf = StratifiedKFold(
                n_splits=5,
                random_state=42,
                shuffle=True
            )

            model_scores = cross_validate(
                estimator=model,
                cv=skf,
                X=X_train,
                y=y_train,
                scoring=metrics
            )

            scores[(n_estimator,max_depth,min_samples_leaf)] = model_scores

for n_estimator in n_estimators:
    for max_depth in max_depths:
        for min_samples_leaf in min_samples_leafs:
            print(f"estimators = {n_estimator} | depth = {max_depth} | leaf
samples = {min_samples_leaf}")
            curr_dict = scores[(n_estimator,max_depth,min_samples_leaf)]
            for metric in metrics:
                mean_score = curr_dict[f"test_{metric}"].mean()
                print(f"Metric : {metric} | Mean Score : {mean_score}")

new_df.loc[(n_estimator,max_depth,min_samples_leaf),metric]=mean_score
```

The code above gave rise to the following table.

n_estimators	max_depths	min_samples_leafs	recall	precision	f1	accuracy
100	3	5	0.84	0.49	0.62	0.72
200	3	20	0.83	0.50	0.62	0.73
200	3	2	0.83	0.49	0.61	0.72
50	3	10	0.83	0.49	0.61	0.72
100	3	20	0.83	0.49	0.62	0.73
200	3	5	0.83	0.50	0.62	0.73
50	3	2	0.83	0.49	0.62	0.73
300	3	20	0.83	0.50	0.62	0.73
100	3	10	0.83	0.50	0.62	0.73
200	3	10	0.83	0.50	0.62	0.73
300	3	2	0.83	0.50	0.62	0.73
300	3	10	0.83	0.50	0.62	0.73
300	3	5	0.83	0.50	0.62	0.73
50	3	20	0.83	0.50	0.62	0.73
50	3	5	0.83	0.50	0.62	0.73
100	3	2	0.82	0.50	0.62	0.74
200	5	10	0.82	0.52	0.63	0.75
50	5	5	0.82	0.52	0.63	0.75
100	5	20	0.82	0.52	0.63	0.75
300	5	10	0.82	0.52	0.63	0.75
200	5	20	0.81	0.52	0.63	0.75
50	5	2	0.81	0.52	0.63	0.75
100	5	2	0.81	0.52	0.63	0.75
300	5	20	0.81	0.52	0.63	0.75
300	5	5	0.81	0.52	0.63	0.75
100	5	5	0.81	0.52	0.63	0.75
200	5	2	0.81	0.52	0.63	0.75
300	5	2	0.81	0.52	0.63	0.75
200	5	5	0.81	0.52	0.63	0.75
100	5	10	0.81	0.51	0.63	0.75
50	5	10	0.81	0.51	0.63	0.75
50	5	20	0.81	0.52	0.63	0.75
100	7	10	0.81	0.53	0.64	0.76
200	7	20	0.81	0.53	0.64	0.76
100	7	20	0.81	0.53	0.64	0.76
300	7	5	0.80	0.53	0.64	0.76
300	7	20	0.80	0.53	0.64	0.76
200	7	10	0.80	0.53	0.64	0.76
50	10	20	0.80	0.54	0.64	0.76
50	7	10	0.80	0.53	0.63	0.76
50	7	5	0.80	0.53	0.64	0.76
300	7	10	0.80	0.53	0.63	0.76
100	7	5	0.80	0.53	0.64	0.76
50	7	20	0.79	0.52	0.63	0.75
200	7	5	0.79	0.53	0.64	0.76
50	7	2	0.79	0.53	0.64	0.76
300	10	20	0.79	0.53	0.64	0.76
100	7	2	0.79	0.53	0.64	0.76
200	10	20	0.79	0.53	0.64	0.76
200	7	2	0.79	0.53	0.64	0.76
300	7	2	0.79	0.54	0.64	0.76
100	10	20	0.79	0.53	0.64	0.76
100	10	10	0.78	0.54	0.64	0.77
50	10	10	0.77	0.55	0.64	0.77
300	10	10	0.77	0.54	0.64	0.77
200	10	10	0.77	0.54	0.64	0.77
300	10	5	0.76	0.55	0.64	0.77
200	10	5	0.76	0.55	0.64	0.77
100	10	5	0.75	0.55	0.64	0.77
50	10	5	0.75	0.55	0.63	0.77
100	10	2	0.73	0.56	0.64	0.78
50	10	2	0.73	0.56	0.64	0.78
200	10	2	0.73	0.56	0.63	0.77
300	10	2	0.73	0.56	0.63	0.78

The table above clearly depicts the robustness of ensemble methods such as the RandomForestClassifier. At a depth of 5 with 10 samples required in the leaf node and tree estimators numbering 200, the recall is ~0.82, which implies that out of the 2000 customers at risk of churn, 1640 of them would be identified correctly, saving ~\$98,400 in revenue @ \$60 average monthly charge, with a precision of 0.52, meaning 48% of all customers predicted to churn would be false positives. That is a 1% reduction compared to the balanced decision tree classifier. At a depth of 7 with 10 samples in the leaf node and estimator count of 100, although recall drops by a percentage to ~0.81, precision increases to ~0.53, implying lesser false positives.

Next, an XGBoost model will be trained, for further refinement.

XGBoost

An XGBoostClassifier model was trained on the data with simple StratifiedKFold sampling without any SMOTE techniques and cross validation was used.

```
n_estimators = [100, 200]
learning_rates = [0.003, 0.009, 0.01, 0.03]
max_depths = [5, 7]
min_child_weights = [5, 10, 20]
metrics = ["recall", "precision", "f1", "accuracy"]

n_rows = len(n_estimators) * len(learning_rates) * len(max_depths) *
len(min_child_weights)
n_cols = len(metrics)

new_arr = np.zeros((n_rows, n_cols))
new_df = pd.DataFrame(
    new_arr,
    index = pd.MultiIndex.from_product(
        [n_estimators, learning_rates, max_depths, min_child_weights],
        names = ["n_estimators", "learning_rate", "max_depth",
"min_child_weights"]
    ),
    columns = metrics
)

models = {}

for n_estimator in n_estimators:
    for learning_rate in learning_rates:
        for max_depth in max_depths:
            for min_child_weight in min_child_weights:

                model = XGBClassifier(
                    n_estimators = n_estimator,
                    learning_rate = learning_rate,
                    max_depth = max_depth,
                    min_child_weight = min_child_weight,
                    subsample = 0.8,
                    colsample_bytree = 0.7,
                    scale_pos_weight = 3,
                    random_state = 42,
                    verbosity = 1
                )

                skf = StratifiedKFold(
                    n_splits = 5,
```

```

        shuffle = True,
        random_state = 42
    )

    scores = cross_validate(
        estimator = model,
        X = X_train,
        y = y_train,
        scoring = metrics,
        cv = skf
    )

models[(n_estimator, learning_rate, max_depth, min_child_weight)] = scores

for n_estimator in n_estimators:
    for learning_rate in learning_rates:
        for max_depth in max_depths:
            for min_child_weight in min_child_weights:
                print(f"n_estimators : {n_estimator}, | learning_rate :
{learning_rate} | max_depth : {max_depth} | min_child_weight :
{min_child_weight}")
                curr_dict =
models[(n_estimator, learning_rate, max_depth, min_child_weight)]
                for metric in metrics:
                    mean_score = curr_dict[f"test_{metric}"].mean()
                    print(f"Metric : {metric} | Mean Score : {mean_score}")

new_df.loc[(n_estimator, learning_rate, max_depth, min_child_weight), metric] =
mean_score

print()

new_df.to_csv(base_dir / "data" / "XGBC_DiffParams.csv")

```

The code above gave rise to a table, a filtered version of which is shown below.

n_estimator	learning_rate	max_depth	min_child_weight	recall	precision	f	accuracy
100	0.003	7	5	0.85	0.50	0.63	0.73
100	0.009	5	5	0.83	0.51	0.63	0.74
100	0.009	5	10	0.83	0.51	0.63	0.74
100	0.009	5	20	0.83	0.51	0.63	0.74
200	0.01	5	20	0.83	0.52	0.64	0.75
200	0.009	5	10	0.83	0.52	0.64	0.75
100	0.009	7	20	0.83	0.51	0.63	0.74
100	0.01	5	5	0.83	0.51	0.63	0.74
100	0.01	5	10	0.83	0.51	0.63	0.74
100	0.01	5	20	0.83	0.51	0.63	0.74
200	0.009	5	5	0.83	0.52	0.64	0.75
200	0.01	5	5	0.83	0.52	0.64	0.75
100	0.01	7	20	0.83	0.51	0.63	0.75
200	0.01	5	10	0.83	0.52	0.64	0.75
200	0.003	7	5	0.82	0.52	0.63	0.75
100	0.009	7	10	0.82	0.52	0.63	0.75
100	0.01	7	10	0.82	0.52	0.64	0.75
100	0.03	5	20	0.82	0.52	0.63	0.75
200	0.003	5	5	0.83	0.51	0.63	0.74
200	0.003	5	10	0.83	0.51	0.63	0.74
200	0.003	5	20	0.84	0.50	0.63	0.74
200	0.009	7	20	0.82	0.52	0.64	0.75
200	0.003	7	10	0.83	0.51	0.63	0.75
200	0.003	7	20	0.83	0.51	0.63	0.74
100	0.03	5	5	0.82	0.52	0.64	0.75
200	0.01	7	20	0.82	0.52	0.64	0.75
200	0.009	5	20	0.83	0.51	0.64	0.75

The filtered table above shows that for an `n_estimator` value of 100, `learning_rate` of 0.003, `max_depth` of 7 and `min_child_weight` of 5, we have a significant improvement in recall at ~0.85 with minimal the precision going down to ~0.50. While this implies that there are many false positives as 50% of the predicted churners will not actually be customers at risk of churn, the high recall value of ~0.85 justifies that effect, as 85% of all churners will be correctly identified. This means that in the scenario that had been talked about, with 2000 total actual churners 1700 of them would actually be identified and there will be a chance to retain them using retention efforts, which if successful, will lead to a revenue saving of \$102,000 per month @ \$60 average monthly charges. This will easily cover the expenses that will be an outcome of retention efforts towards non-churn-risk customers.

LightGBM

An `LGBMClassifier` model was trained on the data using 5 fold cross validation.

```
n_estimators = [150, 200]
learning_rates = [0.01, 0.03, 0.09]
max_depths = [5, 7]
min_child_samples = [5, 10, 20]
num_leaves = [31, 62, 124]
metrics = ["recall", "precision", "f1", "accuracy"]

n_rows = len(n_estimators) * len(learning_rates) * len(max_depths) *
len(min_child_samples) * len(num_leaves)
n_cols = len(metrics)

new_arr = np.zeros((n_rows, n_cols))
new_df = pd.DataFrame(
    new_arr,
    index = pd.MultiIndex.from_product(
        [n_estimators, learning_rates, max_depths, min_child_samples,
num_leaves],
        names = ["n_estimators", "learning_rate", "max_depth",
"min_child_samples", "num_leaves"]
    ),
    columns = metrics
)

models = {}

mix = 1

for n_estimator in n_estimators:
    for learning_rate in learning_rates:
        for max_depth in max_depths:
            for min_child_sample in min_child_samples:
                for num_leaf in num_leaves:

                    print(f"\nModel Set {mix}\n")
                    print(f"estimators : {n_estimator} | learning_rate :
{learning_rate} | max_depth : {max_depth} | min_child_samples :
{min_child_sample} | num_leaves : {num_leaf}")

                    model = LGBMClassifier(
                        n_estimators = n_estimator,
                        learning_rate = learning_rate,
                        max_depth = max_depth,
                        num_leaves = num_leaf,
                        subsample = 0.8,
                        colsample_bytree = 0.7,
```

```

        scale_pos_weight = 3,
        random_state = 42,
        objective = "binary",
        verbosity = -1

    )

    skf = StratifiedKFold(
        n_splits = 5,
        shuffle = True,
        random_state = 42
    )

    scores = cross_validate(
        estimator = model,
        cv = skf,
        X = X_train,
        y = y_train,
        scoring = metrics
    )

    models[(n_estimator, learning_rate, max_depth,
min_child_sample, num_leaf)] = scores

    mix += 1

for n_estimator in n_estimators:
    for learning_rate in learning_rates:
        for max_depth in max_depths:
            for min_child_sample in min_child_samples:
                for num_leaf in num_leaves:
                    print(f"estimators : {n_estimator} | learning_rate :
{learning_rate} | max_depth : {max_depth} | min_child_samples :
{min_child_sample} | num_leaves : {num_leaf}")
                    curr_dict = models[(n_estimator, learning_rate,
max_depth, min_child_sample, num_leaf)]

                    for metric in metrics:

                        mean_score = curr_dict[f"test_{metric}"].mean()
                        new_df.loc[(n_estimator, learning_rate, max_depth,
min_child_sample, num_leaf), metric] = mean_score
                        print(f"Metric : {metric} | Mean Score :
{mean_score}")

                    print()

new_df.to_csv(base_dir / "data" / "LGBM_DiffParams.csv")

```

Using the above code, a table was obtained. The truncated view of the first few columns is shown:

n_estimators	learning_rate	max_depth	min_child_samples	num_leaves	recall	precision	f1	accuracy
150	0.03	5	5	31	0.81	0.53	0.64	0.75
150	0.03	5	10	31	0.81	0.53	0.64	0.75
150	0.03	5	20	31	0.81	0.53	0.64	0.75
150	0.03	5	5	62	0.81	0.53	0.64	0.76
150	0.03	5	5	124	0.81	0.53	0.64	0.76
150	0.03	5	10	62	0.81	0.53	0.64	0.76

When sorted in descending order according to the recall, the best recall attained was 0.81, which is significantly worse than the XGBoost model, however, the precision is comparatively better. It is sensible enough to consider another model apart from the LGBM model.

CatBoost

The following helper function was written to automate the loading of the data for the CatBoostClassifier model as per its requirement as it prefers pure categorical columns.

```
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np
from pathlib import Path

base_dir = Path(__file__).resolve().parent.parent.parent
data_dir = base_dir / "data"

dataset = data_dir / "telco_customer_churn_clean.csv"

def cat_boost_data(dataset):

    df = pd.read_csv(dataset)
    print(f"Dataset '{dataset}' loaded successfully.\n")

    df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
    print("TotalCharges Column Dropped.\n")

    df.dropna(inplace=True)
    print("All null values were dropped.")

    df.drop(columns=["customer_id"], inplace=True)
    print("customerID column was dropped.\n")

    cat_cols = []
    num_cols = []

    for col in df.columns.to_list():
        if df[col].dtype=="str":
            cat_cols.append(col)
        elif df[col].dtype in ("int", "int64", "float", "float64"):
            num_cols.append(col)
        else:
            print(f"Failed to parse column {col}")

    print("Successfully identified categorical columns.\n")

    X = df.drop(columns=["Churn"])
    y = df["Churn"]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        stratify=y,
        test_size=0.2,
        random_state=42
    )

    print(f"Dataset Split complete.\n")

    print(f"Returned X_train, X_test, y_train, y_test and cat_cols.")
```

```
return X_train, X_test, y_train, y_test, cat_cols
```

Then a CatBoostClassifier was trained using the dataset received from the helper function.

```
import pandas as pd
import numpy as np
from catboost import CatBoostClassifier, cv, Pool
from pathlib import Path
import sys
base_dir = Path().resolve().parent
sys.path.append(str(base_dir / "src"))
from helpers.cat_boost_data import cat_boost_data

dataset = base_dir / "data" / "telco_customer_churn_clean.csv"
X_train, X_test, y_train, y_test, cat_cols = cat_boost_data(dataset)

iterations = [100, 200, 300]
learning_rates = [0.003, 0.009, 0.03]
depths = [5, 7]
l2_leaf_regs = [2.0, 3.0, 4.0]
metrics = ["test-Recall-mean", "train-Recall-mean"]

n_rows = len(iterations) * len(learning_rates) * len(depths) *
len(l2_leaf_regs)
n_cols = len(metrics)

new_arr = np.zeros((n_rows, n_cols))
new_df = pd.DataFrame(
    new_arr,
    index=pd.MultiIndex.from_product(
        [iterations, learning_rates, depths, l2_leaf_regs],
        names=["iterations", "learning_rates", "depths", "l2_leaf_regs"]
    ),
    columns=metrics
)

models = {}

no = 1

for iteration in iterations:
    for learning_rate in learning_rates:
        for depth in depths:
            for l2_leaf_reg in l2_leaf_regs:

                train_pool = Pool(
                    data = X_train,
                    label = y_train,
                    cat_features = cat_cols
                )

                params = {
                    "iterations":iteration,
                    "learning_rate":learning_rate,
                    "depth":depth,
                    "l2_leaf_reg":l2_leaf_reg,
                    "loss_function":"Logloss",
                    "eval_metric":"Recall",
                    "custom_metric":[
                        "Precision",
```



```

        "F1",
        "Accuracy"
    ],
    "scale_pos_weight":3,
    "random_seed":42,
    "verbose": 50,
    "task_type":"GPU"
}

cv_results = cv(
    pool = train_pool,
    params = params,
    fold_count = 5,
    stratified = True,
    shuffle = True,
    partition_random_seed = 42,
    early_stopping_rounds = 30
)

models[(iteration,learning_rate,depth,l2_leaf_reg)] =
cv_results

    print(f"Model {no} Trained\n")

    no += 1

for iteration in iterations:
    for learning_rate in learning_rates:
        for depth in depths:
            for l2_leaf_reg in l2_leaf_regs:
                print(f"iteration : {iteration} | learning_rate
{learning_rate} | depth : {depth} | l2_leaf_reg : {l2_leaf_reg}")
                curr_df =
models[(iteration,learning_rate,depth,l2_leaf_reg)]
                for col in curr_df.columns.to_list():
                    if col in metrics:
                        mean_score = curr_df[col].mean()

new_df.loc[(iteration,learning_rate,depth,l2_leaf_reg), col] = mean_score
                print(f"Metric : {col} | Score : {mean_score}")
                print()

new_df.to_csv(base_dir / "data" / "CB_DiffParams.csv")

```

The code above gave rise to a table, a truncated view of which is shown below:

iterations	learning_rates	depths	l2_leaf_regs	test-Recall-mean	train-Recall-mean
200	0.03	5	2	0.83	0.83
300	0.03	5	2	0.83	0.83
200	0.03	5	3	0.83	0.83
300	0.03	5	3	0.83	0.83
200	0.03	5	4	0.83	0.83
300	0.03	5	4	0.83	0.83
100	0.03	5	3	0.83	0.83
200	0.009	5	4	0.83	0.83

The table above shows that at ~200-300 trees, a learning rate of 0.03, a depth of 5 and an L2 regularization score of 2-4, the test-recall comes to a maximum of 0.83, which is still less than what was obtained from the XGBoost model, and as such, the XGBoost model with 0.85 recall and 0.50 precision is being selected for the business case, as it is safer for the company to extend retention efforts towards loyal customers, than risk actually losing customers who are actually at risk.

Final model

Training and Evaluation

From the above done experimentation, it was observed that an XGBClassifier model with the following parameters:

- `n_estimators = 100`
- `learning_rate = 0.003`
- `max_depth = 7`
- `min_child_weights = 5`
- `subsample = 0.8`
- `colsample_bytree = 0.7`
- `scale_pos_weight = 3`
- `random_state = 42` (for reproducibility)

had the highest recall at ~0.85 and an acceptable precision of 0.50. This above model was then formally trained again on the dataset in the same way as was done in the previous iteration, and then the actual testing set was used to quantify its effectiveness.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
import joblib
from sklearn.metrics import recall_score, precision_score, f1_score,
accuracy_score, roc_auc_score, roc_curve
import matplotlib.pyplot as plt
from pathlib import Path
import sys
base_dir = Path().resolve().parent
sys.path.append(str(base_dir / "src"))
from helpers.tree_data import tree_data

data_dir = base_dir / "data"
dataset = "telco_customer_churn_clean.csv"

X_train, X_test, y_train, y_test = tree_data(data_dir / dataset)

xgb_model = XGBClassifier(
    n_estimators = 100,
    learning_rate = 0.003,
    max_depth = 7,
    min_child_weight = 5,
    subsample = 0.8,
    colsample_bytree = 0.7,
    scale_pos_weight = 3,
    random_state = 42,
    verbosity = 2
)

xgb_model.fit(X_train, y_train)

model_dir = base_dir / "model"

joblib.dump(
    xgb_model,
```

```

    model_dir / "xgb_churn_model.pkl"
)

y_pred = xgb_model.predict(X_test)
y_probs = xgb_model.predict_proba(X_test)[:,1]

recall = recall_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_probs)

metrics = {
    "Recall":recall,
    "Precision":precision,
    "F1 score":f1,
    "Accuracy":accuracy,
    "ROC-AUC":roc_auc
}

for metric,score in metrics.items():
    print(f"Metric : {metric} | Score : {score}")

false_pos_rate, true_pos_rate, thresholds = roc_curve(
    y_test,
    y_probs
)

plt.figure(figsize=(12.8, 7.2))
plt.plot(
    false_pos_rate,
    true_pos_rate,
    label = f"ROC Curve (AUC = {roc_auc})"
)
plt.plot(
    [0, 1],
    [0, 1],
    linestyle = "--",
    label = "Random Baseline"
)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.tight_layout()
plt.show()

```

The above model, on completely unseen data, was able to obtain the following metrics:

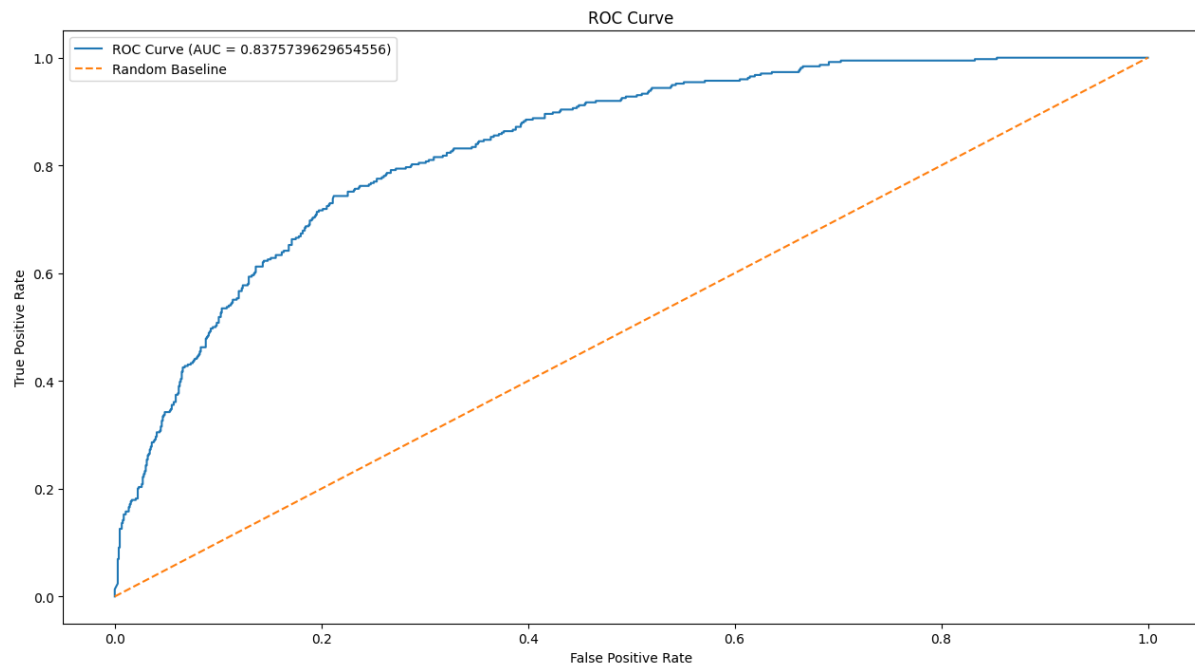
```

Metric : Recall | Score : 0.8315508021390374
Metric : Precision | Score : 0.47480916030534354
Metric : F1 score | Score : 0.6044703595724004
Metric : Accuracy | Score : 0.7107320540156361
Metric : ROC-AUC | Score : 0.8375739629654556

```

The model performed very well on the unseen data and attained a recall score of ~0.83 and a precision score of ~0.47.

The ROC-Curve is shown below.



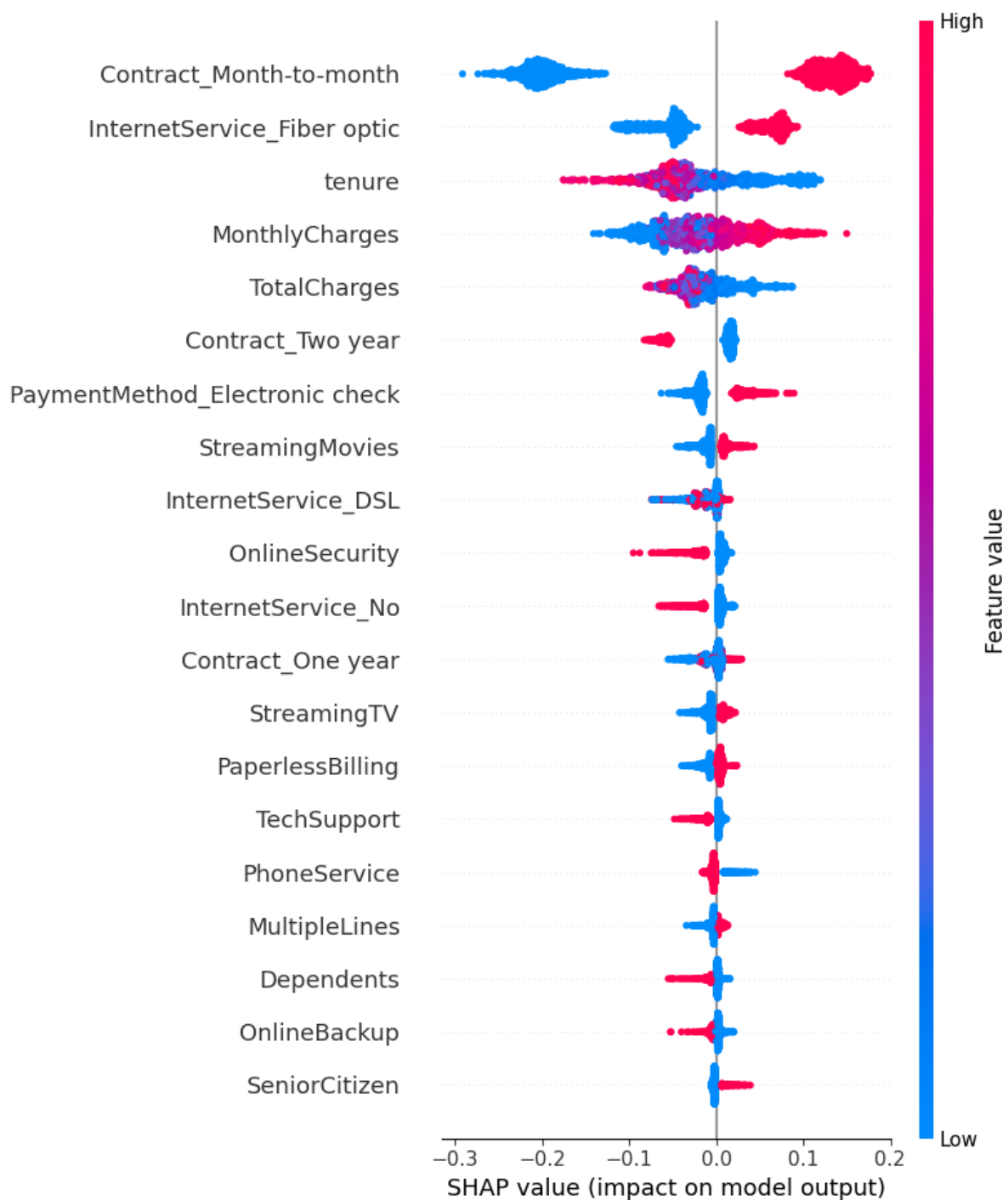
The ROC curve is also very stable and the AUC score of 0.837 implies that if a churner and a non-churner were selected at random, there is an 83.7% chance that the model will assign the higher probability of churn to the customer who was actually a churner.

The Business Impact

In the original dataset there were 1869 churners. Using the model above, ~83% of them, or ~1550 of them, would be identified. As it was discovered earlier (page 19) that among users who churn, using the usual profile, the average monthly charge is ~\$82. So, even with a conservative estimate of monthly charge of \$80, and retention campaign success of 30%, 465 customers can be retained. That would give the company a chance to save upwards of ~\$37,000 per month in revenue loss. This amount would easily cover the cost of retention efforts on loyal customers who were misclassified as churn-risk, and could actually improve loyalty as any churn efforts, such as discounts, offers or bundles, might seem like an unexpected surprise.

Feature Importance Revisited

When it comes to feature importance for the model, the following SHAP visual clearly explains the dominating features.



The SHAP chart above clearly indicates that having a month-to-month plan, fiber optic connection, low tenure, high monthly charges while paying via electronic check and not having any add-on services are correlated with churn, with the first 4 features in the SHAP chart contributing heavily to churn and retention.

Streamlit Prediction App with AI Generated Insights

The SHAP Features

A helper function was written, which helps extract the SHAP waterfall chart for easy interpretability behind the reasons for the prediction.

```
import pandas as pd
import numpy as np
import shap
import matplotlib.pyplot as plt

def extract_top_reasons(dataframe:pd.DataFrame,model):

    explainer = shap.TreeExplainer(model)

    shap_values = explainer(dataframe)

    fig, ax = plt.subplots(figsize=(12,6))

    shap.waterfall_plot(shap_values[0], show=False)

    return fig
```

The AI Generated Insights

Another function was written which uses the “llama-3.3-70b-versatile” model to generate insights based on the risk factors, if and only if the model predicts a churn.

```
from groq import Groq

import streamlit as st

client = Groq(
    api_key=st.secrets["GROQ_API_KEY"]
)

def generate_churn_explanation(
    probability,
    good_signs,
    bad_signs
):

    with open(
        "src/ai_gen/prompt.txt",
        "r",
        encoding="utf-8"
    ) as file:

        prompt_temp = file.read()

    prompt = prompt_temp.format(
        probability=probability,
        good_signs=good_signs,
        bad_signs=bad_signs,
    )

    response = client.chat.completions.create(
```

```

model="llama-3.3-70b-versatile",

messages=[

    {
        "role": "system",
        "content":
            "You are a telecom churn analyst."
    },

    {
        "role": "user",
        "content": prompt
    }

],

temperature=0.5

)

return response.choices[0].message.content

```

The Prompt

You are a telecom customer retention analyst.

You have read the following insights made by the analytics team:

=====
 Analysis revealed that churn was strongly associated with:

1. Monthly subscriptions, with longer duration subscriptions having a lesser rate of churn.
 2. Higher monthly charges, suggesting a mismatch in customer expectation, value perceived, and customer needs.
 3. Tenure, with lower tenured customers being more likely to churn.
 4. Fiber optic internet connection, pointing to possible connectivity issues and/or high cost compared to value.
 5. Not having associated value added services such as Online Security or Tech Support.
 6. Electronic check payments, possibly due to a problem with the payment gateway.
- =====

You have also read the following suggestions made by the consulting team for the business as a whole:

- =====
 Immediate Recommendations
1. Send a feedback form initially via mail to all present customers asking on a scale of 1 to 5 how they would rate their experience or satisfaction regarding different services availed by them, followed by Whatsapp (if registered) or calls till we get the highest amount of response possible.
 2. A separate feedback call to be made to churned customers to identify key areas where they faced problems that ultimately made them churn.
 3. Employees that onboard customers should be given training in batches to make sure that their understanding about the products and services are adequate and up to date so that they will be better able to assist customers.

4. A specialized triage team should be assembled, comprised of employees with deep understanding about the products and services, so that they might be able to analyse the feedback responses.
 5. A special discount can be given to customers opting for fiber optic connections along with add-ons in the one- and two-year contract types.
 6. The billing team must assess the payment gateway to identify any problems customers are facing in electronic check payments and those receiving paperless billing.
 7. A feedback section must appear after payment that lets the customer rate how easy it was to make the payment, and if any problems were faced.
- =====

You are told the following pointers:

- =====
1. There are three contract types, namely, month-to-month, one-year and two-year, with month-to-month customers being more likely to churn and, one- and two-year customers being more likely to be retained.
 2. Customers paying via electronic check have had high churn
 3. The average safe charge for a fiber optic connection is less than 93 and for DSL it is less than 61
 4. Customers with less than 12 tenure are risky
 5. Not having online security is risky
 6. Paying via electronic check is risky
 7. VERY IMPORTANT: Churn probability less than 41% is safe. 41% to 50% is minimal to moderate risk, and more than 50% is definitely risky
- =====

Your task is to analyze the profile of a customer with the following features:

Churn Probability: {probability:.2f}%
Risk factors: The customer is a {bad_signs}
Good signs: The good signs are that the customer {good_signs}

Generate a concise, professional, business-oriented churn assessment report using:

1. Churn probability
2. Customer information
3. Prior analysis and business retention insights

=====

OUTPUT RULES

=====

1. Do NOT explain your reasoning process.
 2. Do NOT describe what you are doing.
 3. Do NOT mention prompt instructions.
 4. Do NOT mention "based on the provided probability".
 5. Do NOT generate generic recommendations.
 6. Keep the response concise, professional, analytical, and customer-specific.
 7. Return ONLY the final report.
- =====

OUTPUT FORMAT

=====

Risk Assessment

Explain why this customer MIGHT BE likely to churn.

Key Churn Drivers

Explain the most important churn contributors (if any) relevant to this customer.

Recommended Retention Actions

Provide between 1 and 5 concise customer-specific retention recommendations (if required).

The Streamlit App

```

import streamlit as st
import pandas as pd
import numpy as np
import joblib
import plotly.graph_objects as go
from pathlib import Path
import sys
base_dir = Path(__file__).resolve().parent
sys.path.append(str(base_dir / "src"))
from shap_gen.shap_gen import extract_top_reasons
from ai_gen.ai_gen import generate_churn_explanation

# Page configuration
st.set_page_config(
    page_title="Customer Churn Prediction",
    page_icon="📊",
    layout="wide"
)

# Loading the saved model
model = joblib.load("model/xgb_churn_model.pkl")

# Title
st.title("📊 Customer Churn Prediction Dashboard")

# Sidebar section
st.sidebar.header("Customer Features")

# 1
gender = st.sidebar.radio(
    "Gender",
    ["Male", "Female"]
)

gender = 1 if gender == "Male" else 0

# 2
SeniorCitizen = st.sidebar.radio(
    "Senior Citizen",
    ["Yes", "No"]
)

SeniorCitizen = 1 if SeniorCitizen == "Yes" else 0

```

```
# 3
Partner = st.sidebar.radio(
    "Partner",
    ["Yes", "No"]
)

Partner = 1 if Partner == "Yes" else 0

# 4
Dependents = st.sidebar.radio(
    "Dependents",
    ["Yes", "No"]
)

Dependents = 1 if Dependents == "Yes" else 0

# 5
tenure = st.sidebar.slider(
    "Tenure",
    min_value=0,
    max_value=72,
    value=12
)

tenure_input = st.sidebar.number_input(
    "Or Enter Tenure Directly",
    min_value=0.0,
    max_value=72.0,
    value=float(tenure),
    step=1.0
)

tenure = int(round(tenure_input))

# 6
PhoneService = st.sidebar.radio(
    "Phone Service",
    ["Yes", "No"]
)

PhoneService = 1 if PhoneService == "Yes" else 0

# 7
if PhoneService == 0:

    st.sidebar.radio(
        "Multiple Lines",
        ["No"],
        disabled=True
    )

    MultipleLines = 0

else:

    MultipleLines = st.sidebar.radio(
        "Multiple Lines",
        ["Yes", "No"]
    )
```

```
MultipleLines = 1 if MultipleLines == "Yes" else 0

# 8
internet_service = st.sidebar.selectbox(
    "Internet Service",
    [
        "DSL",
        "Fiber optic",
        "No internet service"
    ]
)

InternetService_DSL = 0
InternetService_Fiber_optic = 0
InternetService_No = 0

if internet_service == "DSL":
    InternetService_DSL = 1

elif internet_service == "Fiber optic":
    InternetService_Fiber_optic = 1

else:
    InternetService_No = 1

# Internet dependent features

internet_disabled = internet_service == "No internet service"

def internet_feature(label):

    if internet_disabled:

        st.sidebar.radio(
            label,
            ["No"],
            disabled=True
        )

        return 0

    else:

        value = st.sidebar.radio(
            label,
            ["Yes", "No"]
        )

        return 1 if value == "Yes" else 0

# 9-14
OnlineSecurity = internet_feature("Online Security")
OnlineBackup = internet_feature("Online Backup")
DeviceProtection = internet_feature("Device Protection")
TechSupport = internet_feature("Tech Support")
StreamingTV = internet_feature("Streaming TV")
StreamingMovies = internet_feature("Streaming Movies")

# Contract

contract = st.sidebar.selectbox(
```

```
        "Contract",
        [
            "Month-to-month",
            "One year",
            "Two year"
        ]
    )

    Contract_Month_to_month = 0
    Contract_One_year = 0
    Contract_Two_year = 0

    if contract == "Month-to-month":
        Contract_Month_to_month = 1

    elif contract == "One year":
        Contract_One_year = 1

    else:
        Contract_Two_year = 1

    # Paperless Billing
    PaperlessBilling = st.sidebar.radio(
        "Paperless Billing",
        ["Yes", "No"]
    )

    PaperlessBilling = 1 if PaperlessBilling == "Yes" else 0

    # Payment Method
    payment_method = st.sidebar.selectbox(
        "Payment Method",
        [
            "Bank transfer (automatic)",
            "Credit card (automatic)",
            "Electronic check",
            "Mailed check"
        ]
    )

    PaymentMethod_Bank_transfer = 0
    PaymentMethod_Credit_card = 0
    PaymentMethod_Electronic_check = 0
    PaymentMethod_Mailed_check = 0

    if payment_method == "Bank transfer (automatic)":
        PaymentMethod_Bank_transfer = 1

    elif payment_method == "Credit card (automatic)":
        PaymentMethod_Credit_card = 1

    elif payment_method == "Electronic check":
        PaymentMethod_Electronic_check = 1

    else:
        PaymentMethod_Mailed_check = 1

    # Monthly Charges
    MonthlyCharges = st.sidebar.slider(
        "Monthly Charges",
        min_value=0.0,
```

```

        max_value=200.0,
        value=70.0
    )

MonthlyCharges_input = st.sidebar.number_input(
    "Or Enter Monthly Charges Directly",
    min_value=0.0,
    max_value=200.0,
    value=float(MonthlyCharges),
    step=0.1
)

MonthlyCharges = MonthlyCharges_input

# Total charges
adjustment = MonthlyCharges * tenure * 0.0001422
TotalCharges = (MonthlyCharges * tenure) - adjustment

# DataFrame
input_data = pd.DataFrame({

    "SeniorCitizen": [SeniorCitizen],
    "gender": [gender],
    "Partner": [Partner],
    "Dependents": [Dependents],
    "tenure": [tenure],
    "PhoneService": [PhoneService],
    "MultipleLines": [MultipleLines],

    "InternetService_DSL": [InternetService_DSL],
    "InternetService_Fiber_optic": [InternetService_Fiber_optic],
    "InternetService_No": [InternetService_No],

    "OnlineSecurity": [OnlineSecurity],
    "OnlineBackup": [OnlineBackup],
    "DeviceProtection": [DeviceProtection],
    "TechSupport": [TechSupport],
    "StreamingTV": [StreamingTV],
    "StreamingMovies": [StreamingMovies],

    "Contract_Month-to-month": [Contract_Month_to_month],
    "Contract_One_year": [Contract_One_year],
    "Contract_Two_year": [Contract_Two_year],

    "PaperlessBilling": [PaperlessBilling],

    "PaymentMethod_Bank transfer (automatic)":
[PaymentMethod_Bank_transfer],
    "PaymentMethod_Credit card (automatic)": [PaymentMethod_Credit_card],
    "PaymentMethod_Electronic check": [PaymentMethod_Electronic_check],
    "PaymentMethod_Mailed check": [PaymentMethod_Mailed_check],

    "MonthlyCharges": [MonthlyCharges],
    "TotalCharges": [TotalCharges]

})

# Main section
st.markdown("---")

predict_button = st.button("🚀 PREDICT")

```

```

if not predict_button:

    st.subheader(
        "Select customer features to predict churn probability."
    )

else:

    prediction = model.predict(input_data)[0]

    probability = model.predict_proba(input_data)[0][1]

    probability_percent = round(probability * 100, 2)

    # Gauge chart
    color = "red" if prediction == 1 else "green"

    fig = go.Figure(go.Indicator(
        mode="gauge+number",
        value=probability_percent,

        title={
            "text": "Churn Probability (%)"
        },

        gauge={
            "axis": {
                "range": [0, 100]
            },

            "bar": {
                "color": color
            }
        }
    ))

    st.plotly_chart(
        fig,
        use_container_width=True
    )

    # Prediction text
    if prediction == 1:

        st.error(
            f"⚠️ There is a {round(float(probability_percent), 2)}% chance
the Customer will Churn."
        )

    else:

        st.success(
            f"✅ Customer is likely to stay with a "
            f"{round(100 - float(probability_percent), 2)}% chance."
        )

    # Top reasons for churn
    top_reasons = extract_top_reasons(dataframe = input_data, model = model)

    st.markdown("### SHAP Waterfall plot")

```

```

st.pyplot(top_reasons)

reasons = []

for col in input_data.columns.to_list():
    if input_data[col].iloc[0] == 1:
        reasons.append(col)

risk_factors = ["InternetService_Fiber optic",
                "Contract_Month-to-month",
                "PaymentMethod_Electronic check"]
risk_reasons = ["Fiber optic customer",
                "Month to month customer",
                "Customer paying via electronic check"]
safe_factors = ["Partner", "Dependents", "OnlineSecurity",
                "OnlineBackup", "DeviceProtection",
                "TechSupport", "Contract_One year",
                "Contract_Two year"]
safe_reasons = ["Has partners",
                "Has dependents",
                "Has online security",
                "Has online backup",
                "Has device protection",
                "Has tech support",
                "Has a one year contract",
                "Has a two year contract"]

risks = dict(zip(risk_factors, risk_reasons))
safeties = dict(zip(safe_factors, safe_reasons))

churn_risks = []
churn_safes = []

for risk, risk_reason in risks.items():
    if risk in reasons:
        churn_risks.append(risk_reason)

for safety, safety_reason in safeties.items():
    if safety in reasons:
        churn_safes.append(safety_reason)

risky = ("", ").join(churn_risks)
safe = ("", ").join(churn_safes)

# AI driven insights
if probability_percent > 50:
    response = generate_churn_explanation(
        probability=round(float(probability_percent), 2),
        good_signs=safe,
        bad_signs=risky
    )

    st.markdown("## AI Insights")
    st.markdown(f"!! {response}")

elif probability_percent > 41:
    st.markdown('## Insight')
    st.markdown("⚠️ Although customer is low risk, use caution.\n"
                "Send a feedback form via mail and warm call after"
response.")

```



```
if len(churn_risks)>0:
    st.markdown(f"Risk factors are this customer is a {risky}")
if len(churn_safes)>0:
    st.markdown(f"Postiive points are the customer {safe}")

else:
    st.markdown("## Insight")
    st.markdown("☑ Customer is safe, and not likely to churn.")
    if len(churn_risks)>0:
        st.markdown(f"Risk factors however are this customer is a
{risky}")
    if len(churn_safes)>0:
        st.markdown(f"Postiive points are the customer {safe}")
```

Conclusion

This project was based on a telecom customer churn dataset to analyse customer behaviour patterns in an attempt to identify the major factors responsible for customer churn, and to support data-driven retention strategies through an interactive analytical web-application.

The project began with some minor data cleaning, followed by extensive preprocessing, and exploratory data analysis to uncover relationships between customer demographics, services availed, billing behaviour, contract types, and churn outcomes. Several trends and behavioural patterns associated with customer attrition were successfully identified, helping establish a clearer understanding of the business problem from an analytical perspective.

To quantify churn risk and support predictive analysis, multiple machine learning models were evaluated using business-relevant performance metrics such as recall, precision, F1-score, and accuracy. Special attention was given to handling class imbalance and minimizing false negatives, as failing to identify potentially churn-prone customers could negatively affect retention efforts. Also, extensive efforts were made to avoid multicollinearity for the linear models. The final model was selected based on both predictive performance and interpretability.

An interactive Streamlit dashboard was then developed which will allow the company to predict the probability of a customer churning by dynamically entering the specific features of that customer. The dashboard provides a business-friendly interface where customer attributes can be modified in real time to identify the churners, so that timely intervention may be performed.

To improve interpretability and analytical value, SHAP was integrated into the system to identify the strongest contributing factors behind individual churn predictions. This allowed the application to move beyond simple probability outputs and provide deeper analytical insight into customer risk behaviour.

Additionally, a controlled AI-assisted insight layer was incorporated for higher-risk churn cases. Instead of using generative AI universally, deterministic business logic was prioritized for low-risk customers to maintain consistency and reliability, while AI-generated retention insights were selectively enabled only for customers with elevated churn risk. This hybrid approach ensured that analytical accuracy and business practicality remained the primary focus of the system.

Overall, the project combines:

- Data Analysis for business understanding and trend identification
- Data-backed insights and recommendations for first-line-defence strategy
- Predictive Analytics for churn risk estimation
- Explainable AI for interpretable insights
- Interactive Dashboarding for user engagement
- AI-assisted business recommendations for high-risk customer cases

This project demonstrates the practical application of data analytics, predictive modeling, visualization, and explainable AI within a real-world business context. More importantly, it highlights how analytical systems can be enhanced with AI capabilities while still maintaining transparency, interpretability, and business-focused decision support.